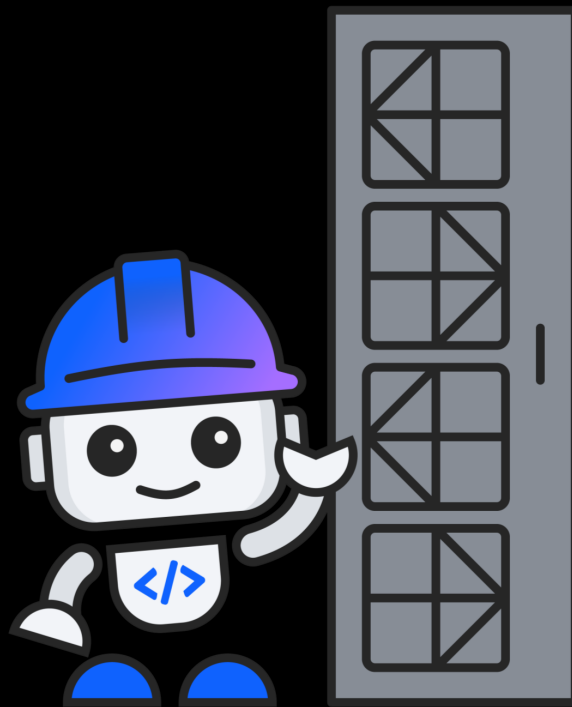


# *BOB Premium Package for Z*



## Table of Contents

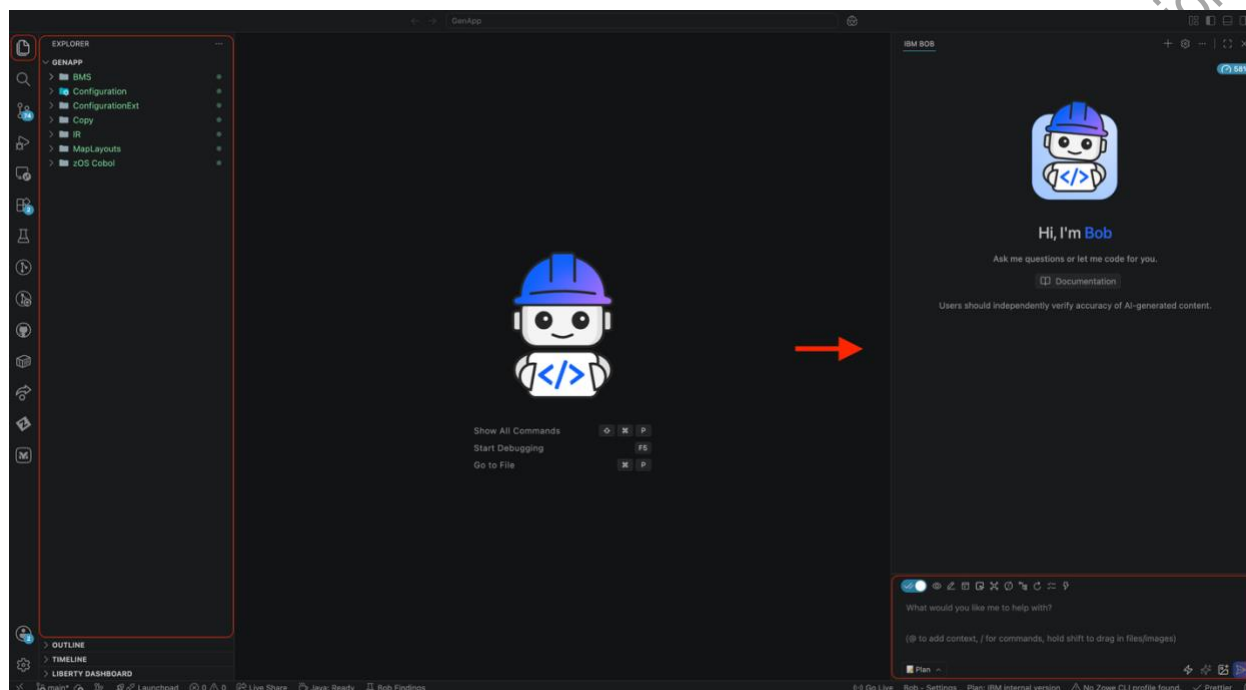
|                                                                            |    |
|----------------------------------------------------------------------------|----|
| <i>Accessing the Environment</i> .....                                     | 3  |
| Bob .....                                                                  | 3  |
| Adding the Sample Application .....                                        | 3  |
| Switching Modes in BOB .....                                               | 3  |
| Auto Approve .....                                                         | 4  |
| <i>Analyze understand, technical documentation &amp; explanation</i> ..... | 5  |
| Scan Workspace .....                                                       | 5  |
| Agent.md .....                                                             | 8  |
| Data Dictionary .....                                                      | 12 |
| Create a useful technical design document .....                            | 15 |
| <i>Impact of Changing a variable</i> .....                                 | 18 |
| Generating analysis documentation of a change .....                        | 18 |
| Analysis of the generated document .....                                   | 23 |
| Implementing the Change .....                                              | 24 |
| <i>Monolithic Cobol refactoring and service integration</i> .....          | 29 |
| Identify Business Rules Functionality .....                                | 29 |
| Check Dependencies .....                                                   | 32 |
| Refactoring business services .....                                        | 34 |
| Add link code to main module .....                                         | 37 |
| <i>CICS Online screens to modern UI Experience</i> .....                   | 39 |
| Understand & analyze insurance application .....                           | 39 |
| Leverage prompt within chat to implement UI. ....                          | 44 |

## Accessing the Environment

### Bob

**ACTION:** Please access the BOB UI. The following image represents how the UI will look.

1. The panel on the left is your file explorer, this is where various artifacts will be generated and the code is located.
2. On the right is the chat window where you will explore various use cases.



### Adding the Sample Application

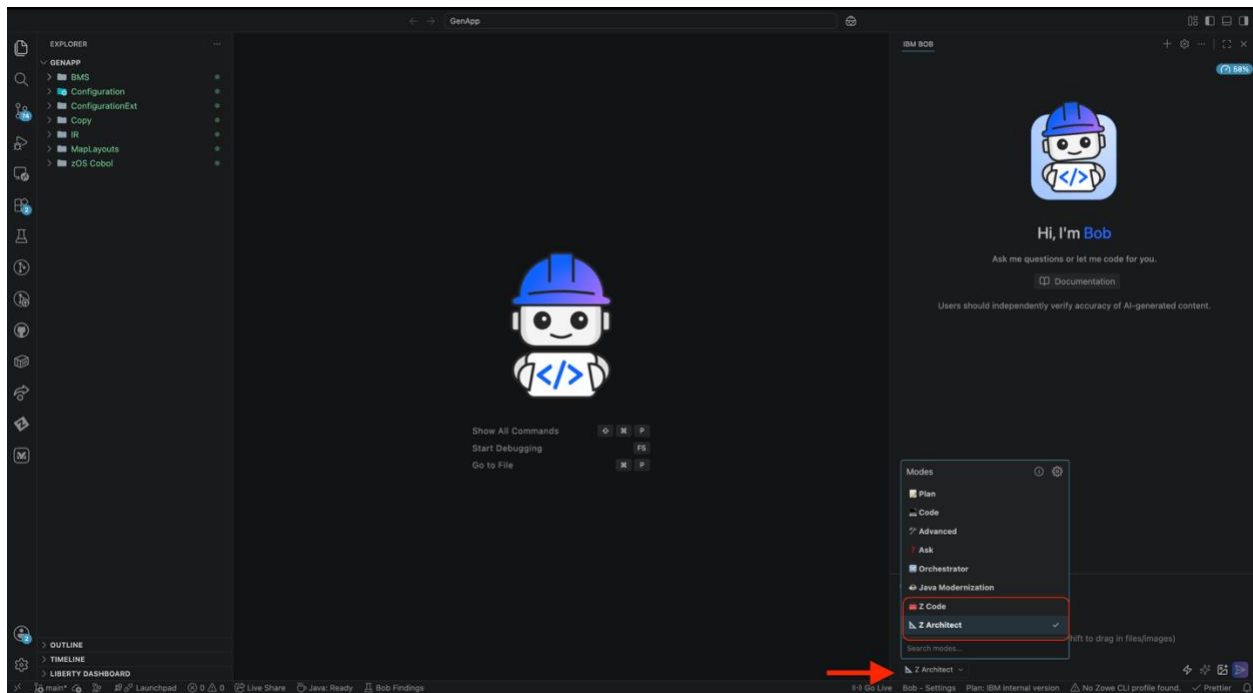
In the case that the code is not available on your environment, or in your workspace. Please let an instructor know.

### Switching Modes in BOB

If you click on the mode's icon on the bottom of the chat window, you will see various modes. To switch to that mode please select the one of interest.

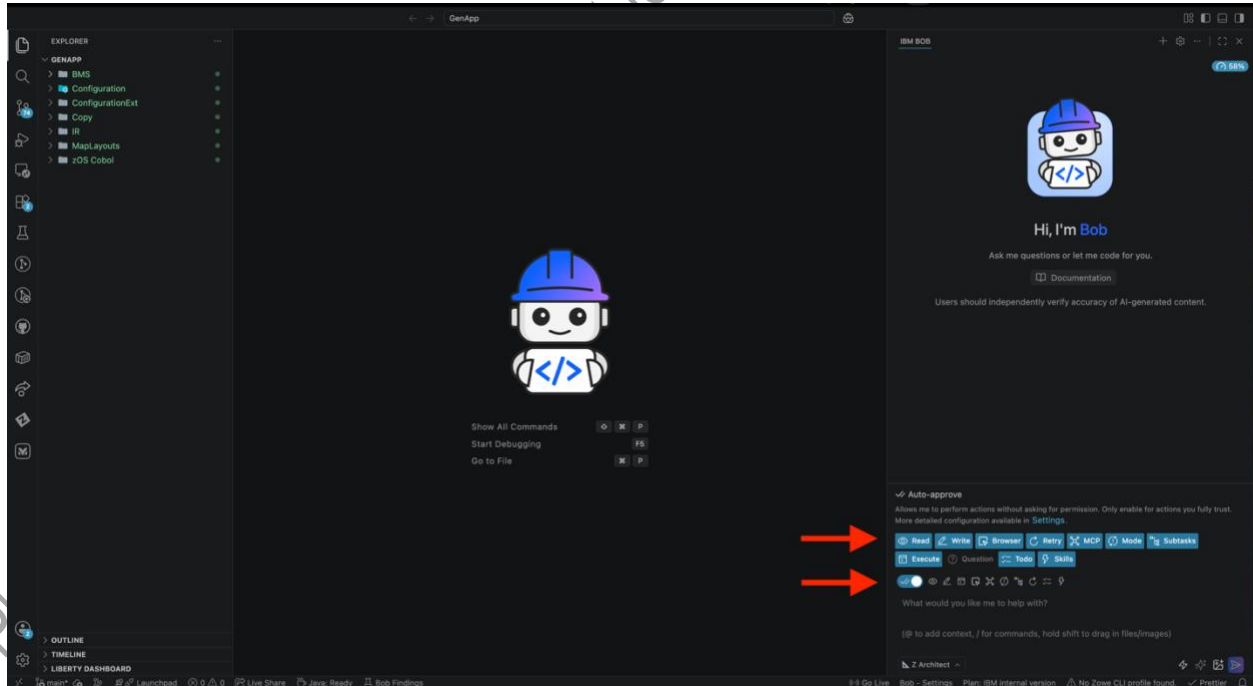
| Mode        | Description                                                                                                              |
|-------------|--------------------------------------------------------------------------------------------------------------------------|
| z-architect | Conduct implementation planning, conduct impact analysis, understand system design, and generate technical documentation |
| z-code      | Explain, change, and modernize your mainframe applications                                                               |

To explore more information on Modes: <https://bob.ibm.com/docs/ide>



## Auto Approve

**Optional:** If multiple related files are listed together in a single approval request, you can click **Approve All** to let BOB process them in one step, rather than approving each file individually.



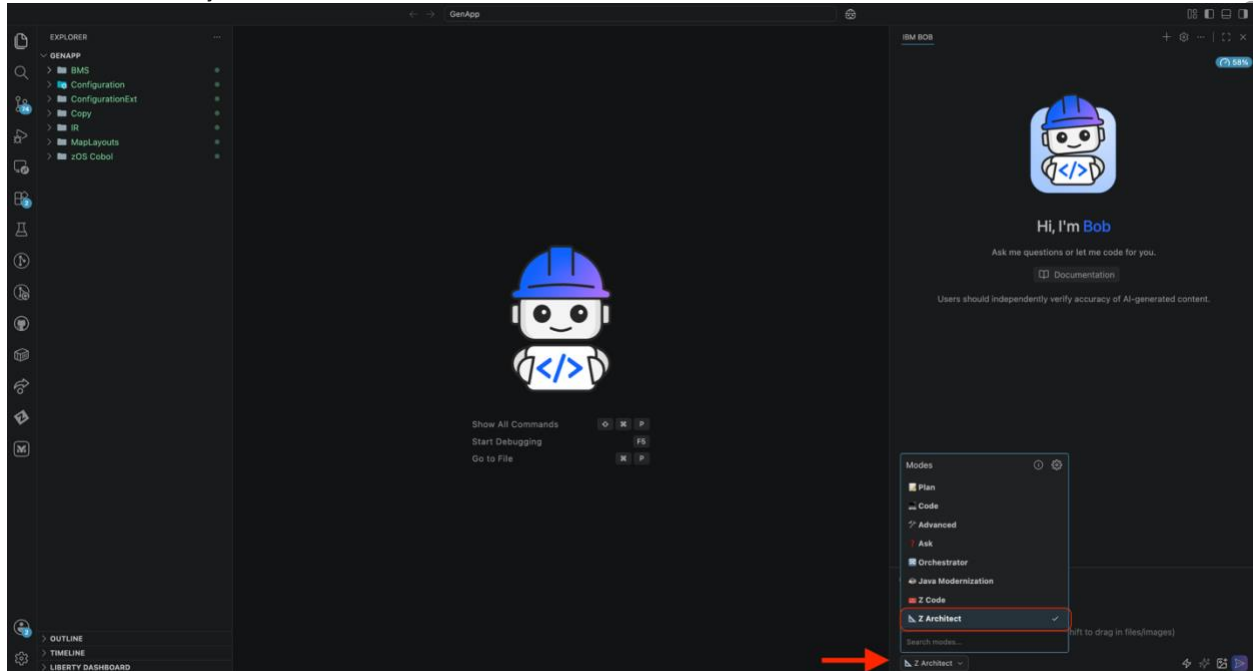


Analyze understand, technical documentation & explanation.

### Scan Workspace

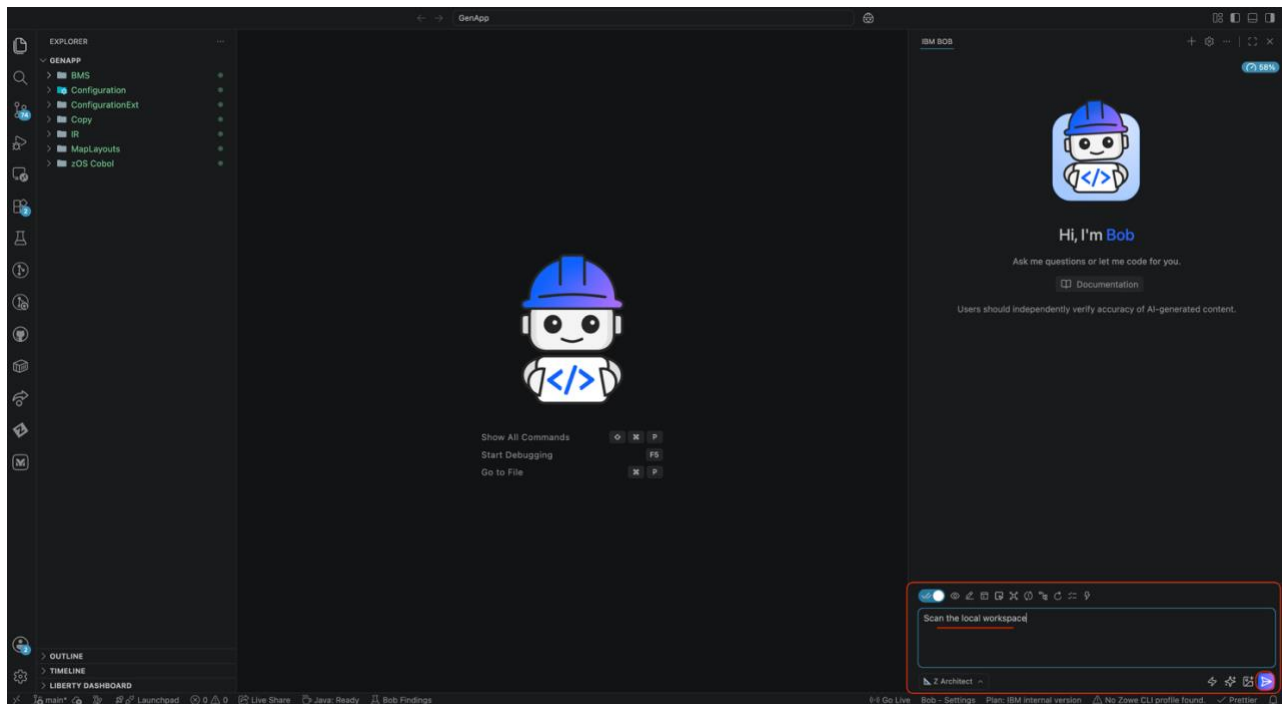
Run Scanner locally to populate metadata at workspace level in local database. Workflow allows to experience the Z Premium package at the workspace level.

**ACTION:** Ensure you are in the Z Architect Mode.



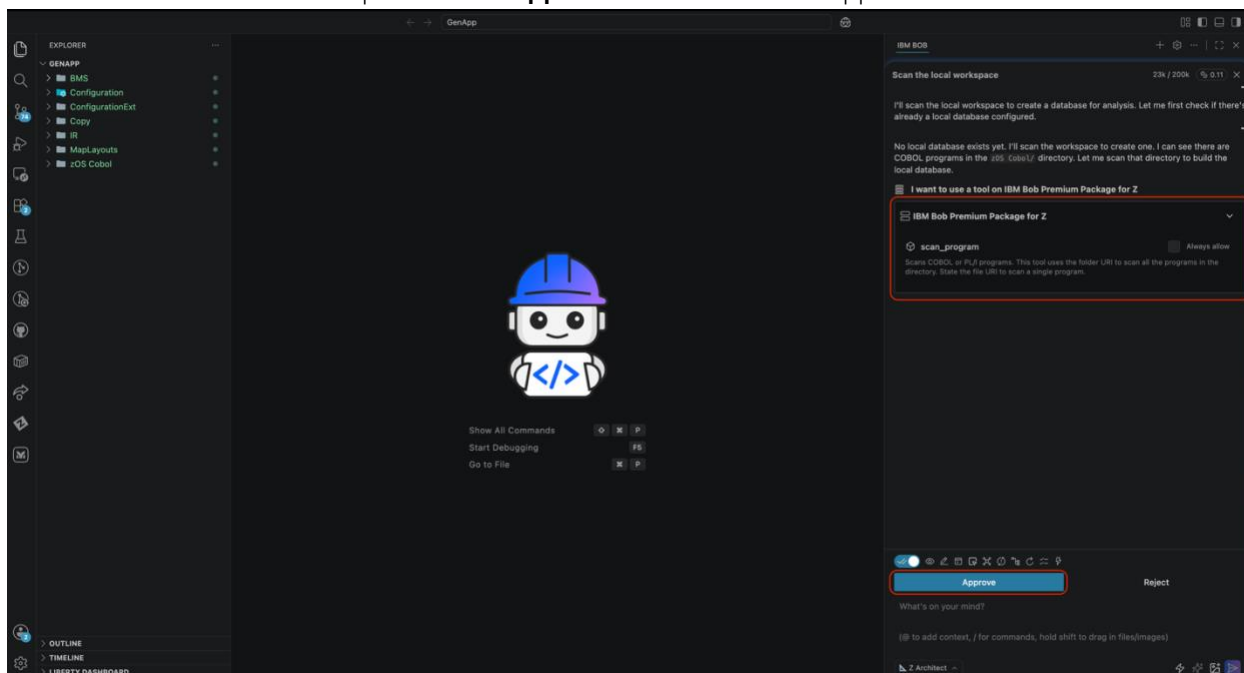
**ACTION:** In the chat window, please paste the following prompt.

Scan the local workspace

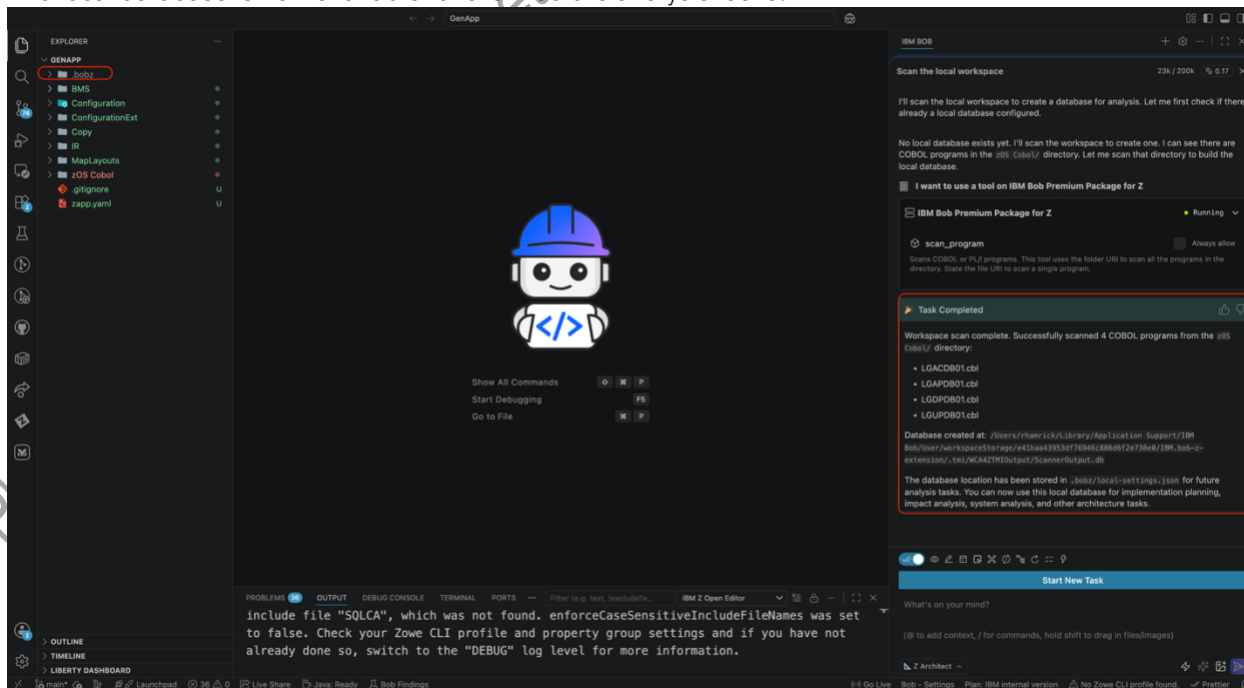


**ACTION:** A tool request dialog will appear for scan\_program. Review the request details, then click Approve to allow BOB to begin the scan. Do not close VS Code while the scan is in progress, the scan must complete fully before BOB can run its analysis queries.

⚠ You may be asked to approve other tool requests in various sequences, these include scan\_program, or other individual file read requests. Click **Approve** for each one as it appears.



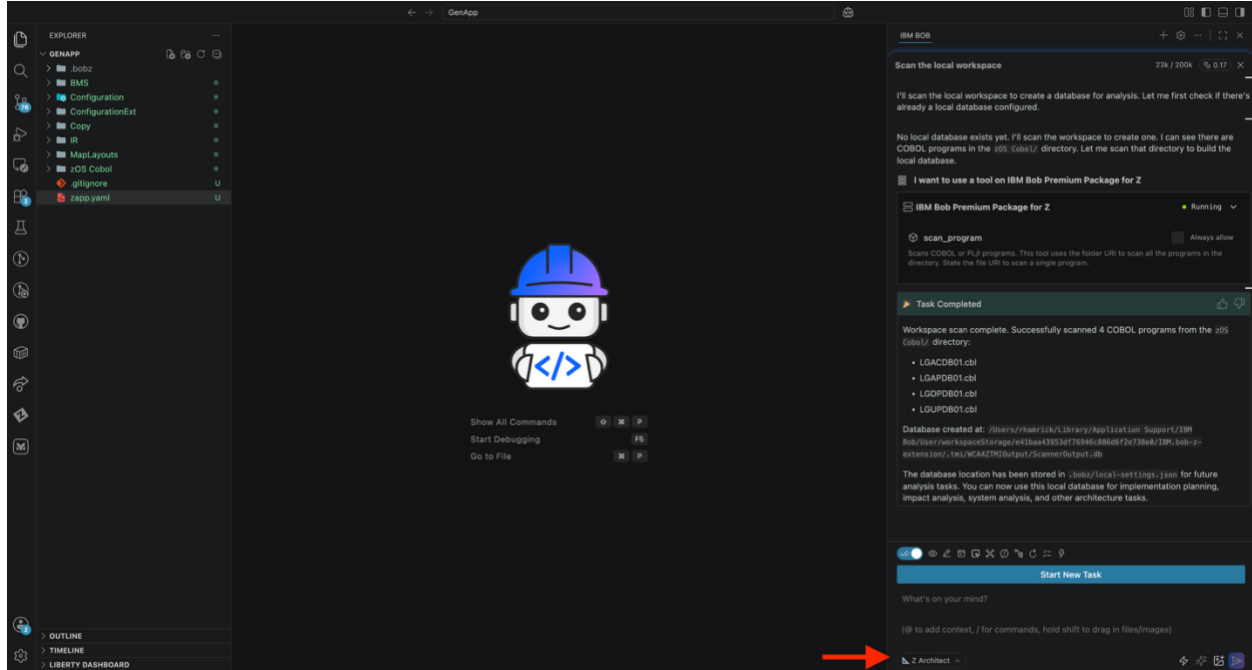
**ACTION:** When completed, the finished scan should resemble the following screen. The local database is now available for architecture analysis tasks.



## Agent.md

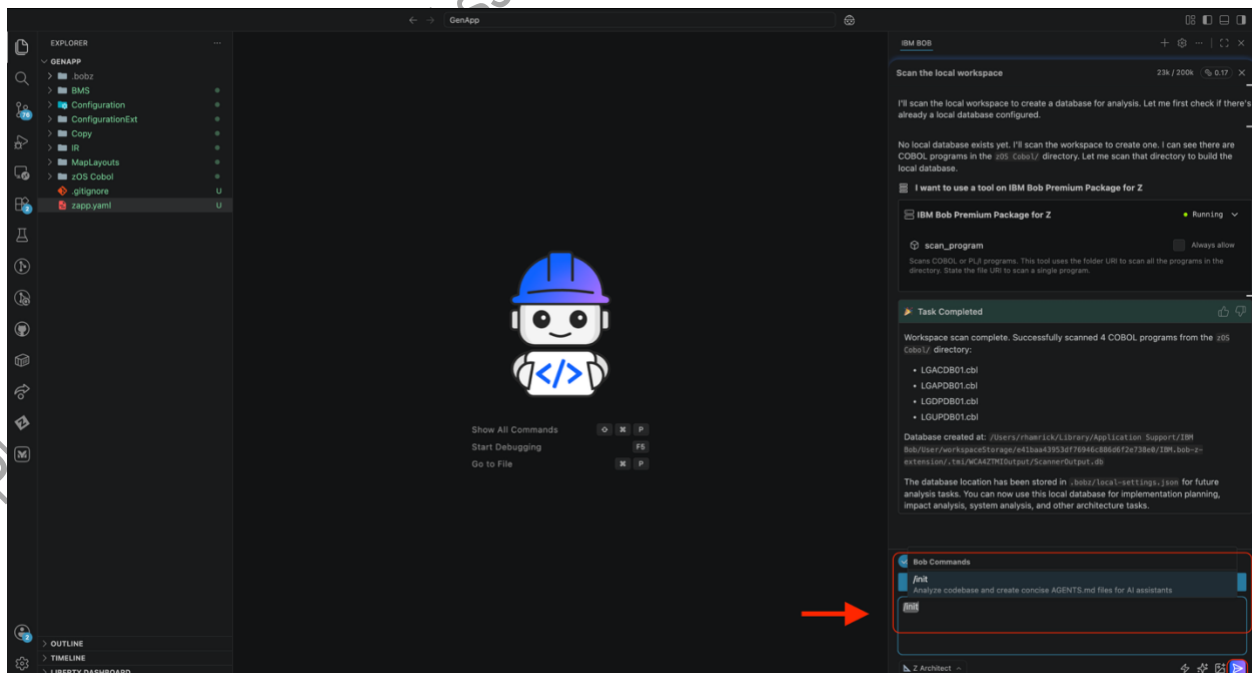
The Agent.md creates a central reference for the agents, containing key artifacts such as file locations and user preferences, which guides the agents in analyzing COBOL programs by capturing details such as workspace type, repository structure, data dictionary location, and other relevant configuration information.

**ACTION:** Please open the BOB UI and ensure you are in **Z Architect Mode**

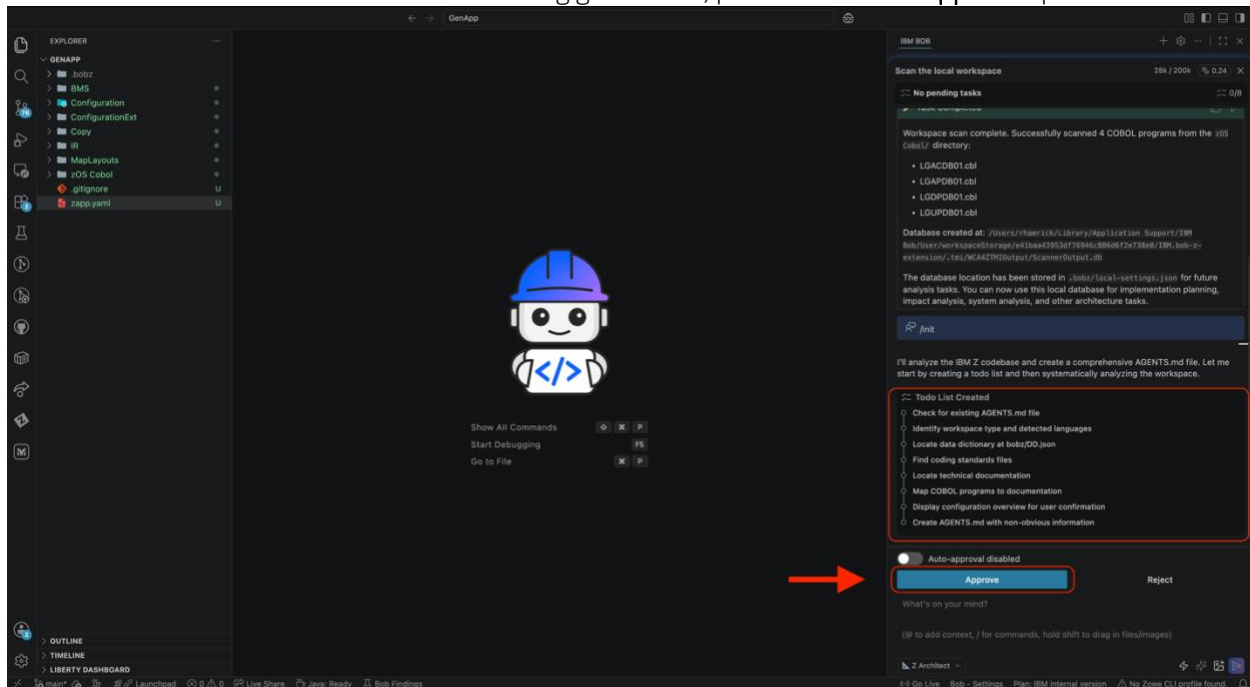


**ACTION:** Let's start by creating the agent.md file. In Z Architect Mode, please type:

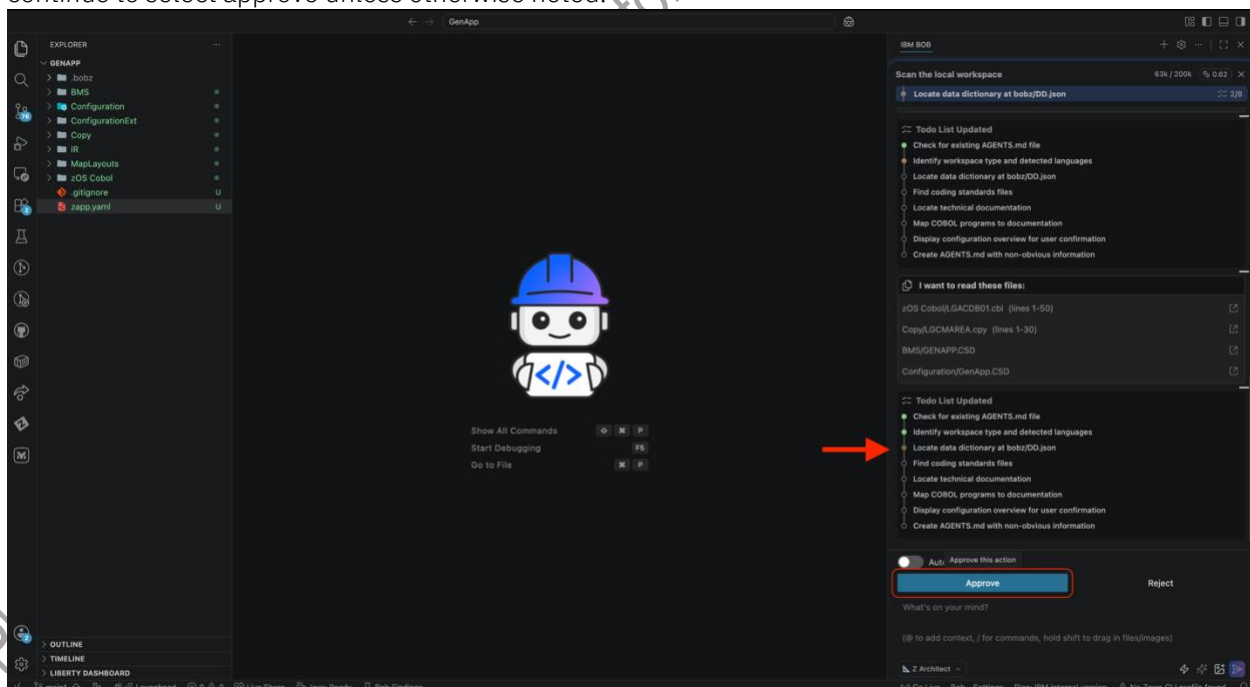
/init



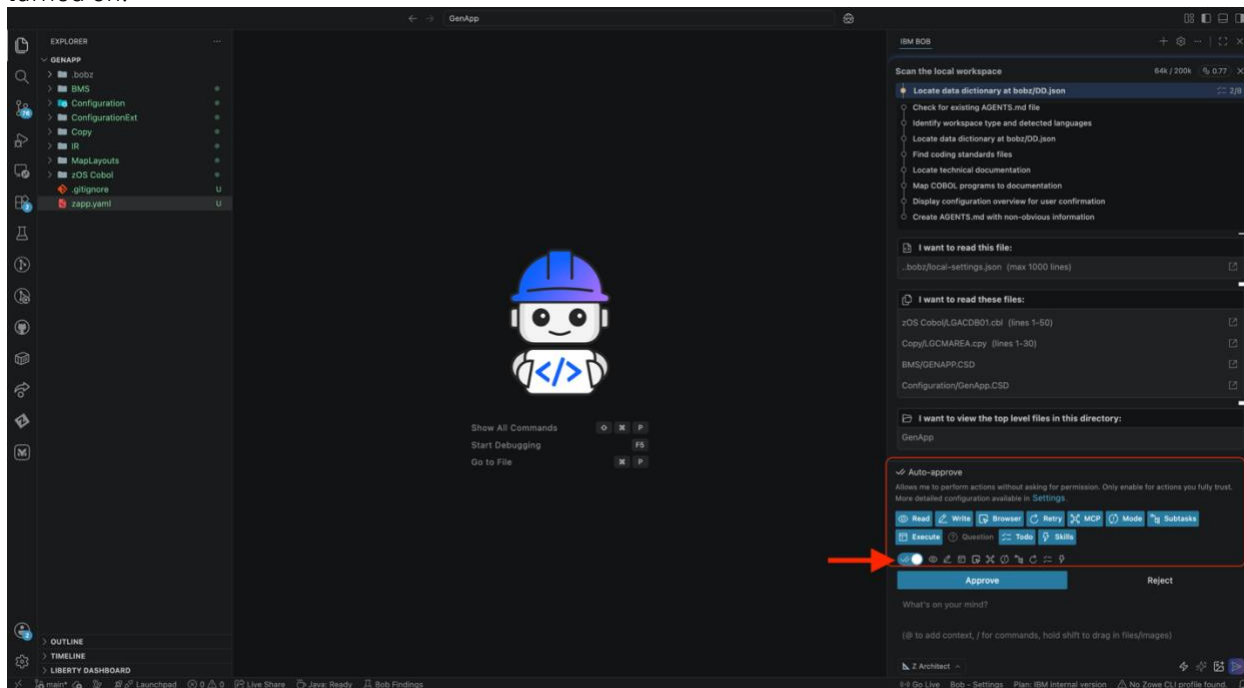
**ACTION:** You'll see BOB create a task list during generation, please select the **approve** option here.



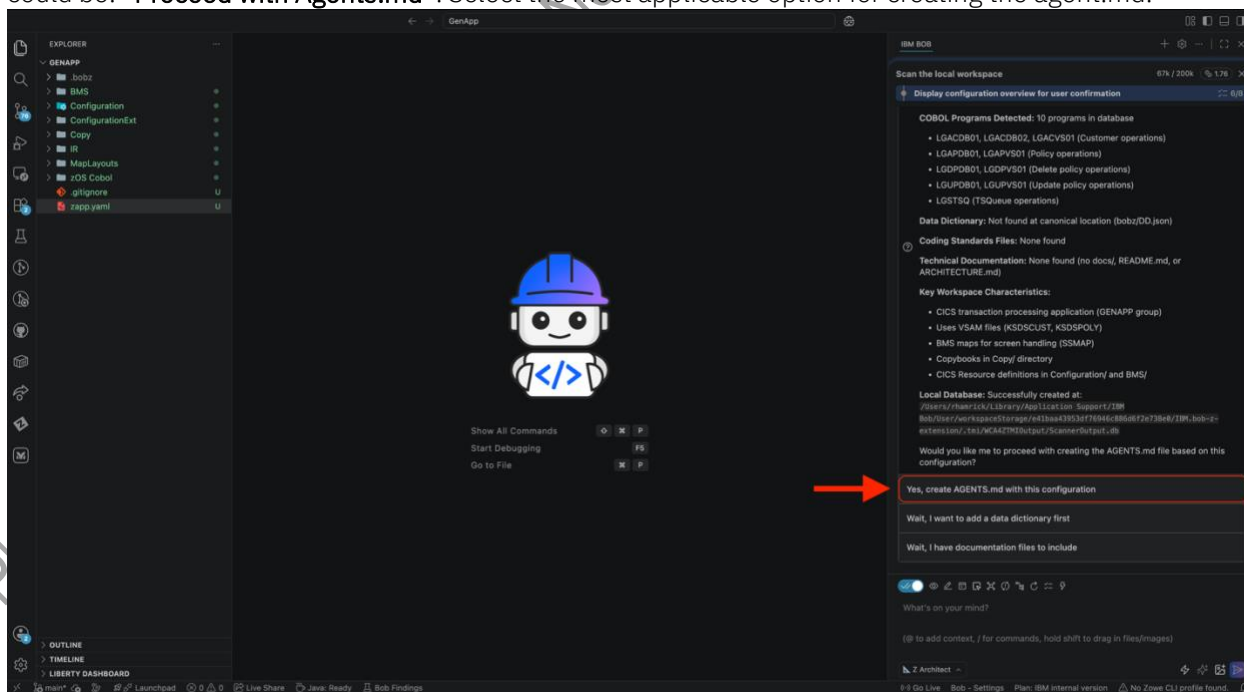
**ACTION:** You'll notice that during the completion of the task each step will ask to be approved. Please continue to select approve unless otherwise noted.



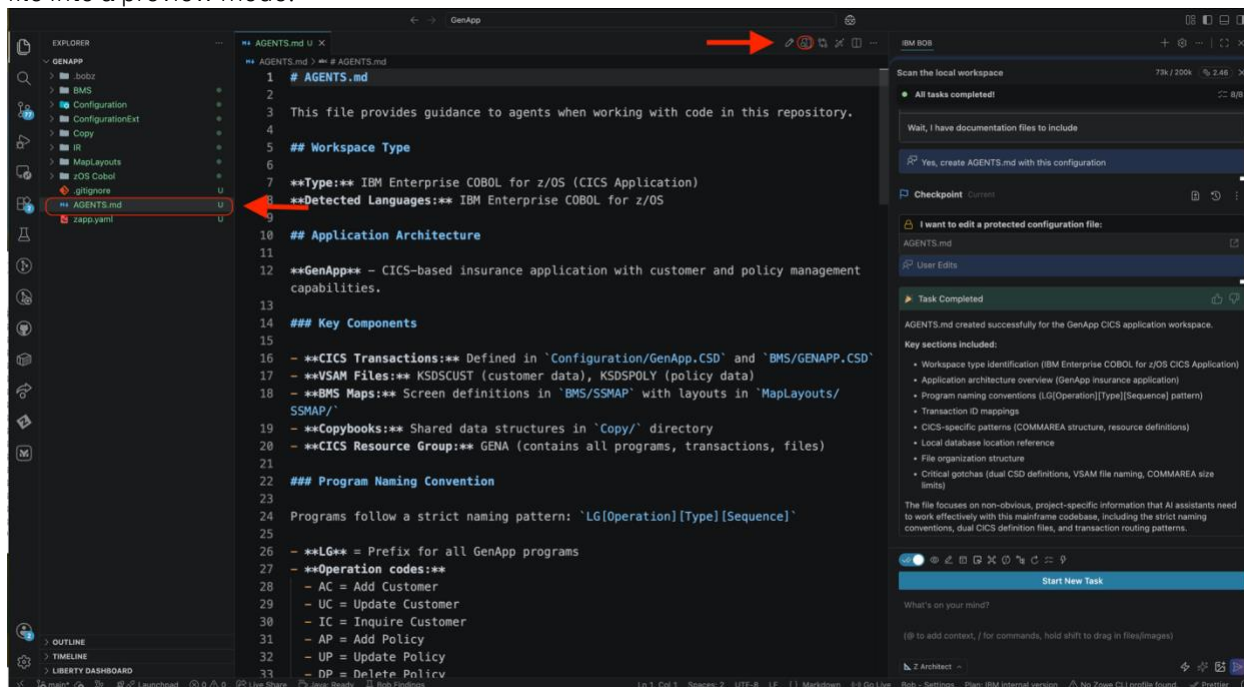
**Optional:** Select auto approve for specific tasks, to limit the amount of approval you will be asked, feel free to turn on auto approval. I would recommend keeping the questions option off. Blue indicates it's been turned on.



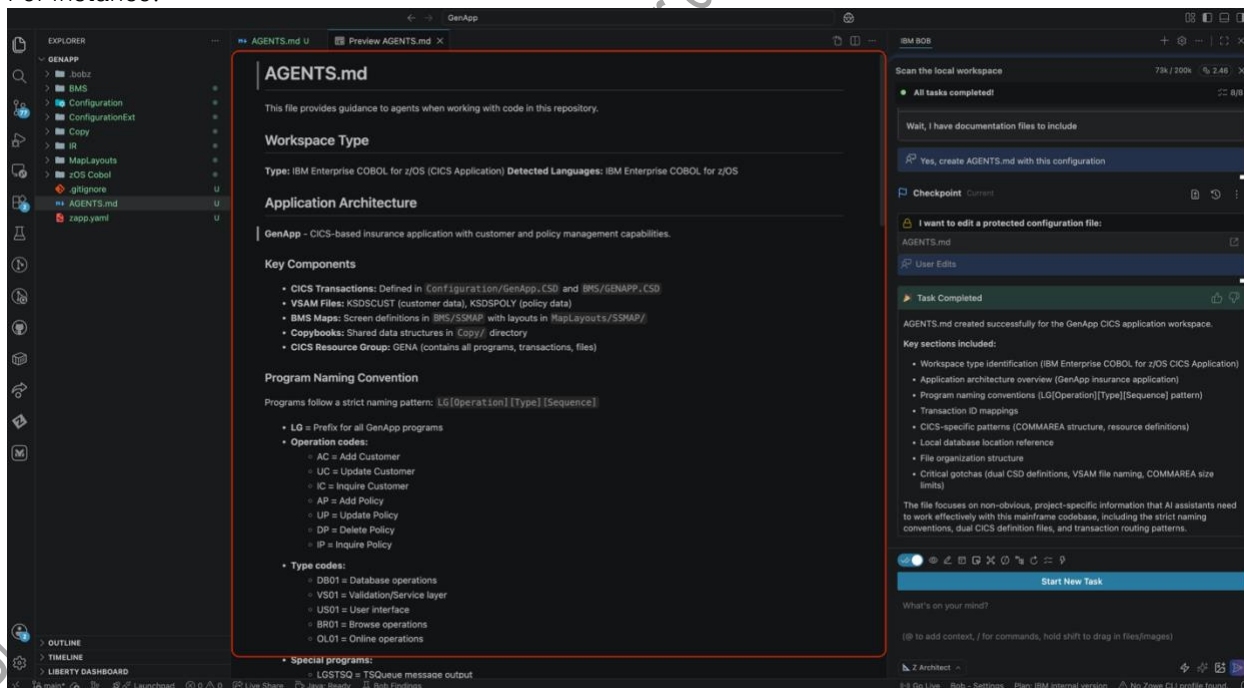
**ACTION:** You may see a different set of options than below. For instance, if available please select “Yes, create bobz/ directory and note the data dictionary location for future use.” Another option you may see, could be: **“Proceed with Agents.md”**. Select the most applicable option for creating the agent.md.



**ACTION:** You'll notice that an AGENT.md file has now been created. Please select the preview open the file into a preview mode.



For instance:



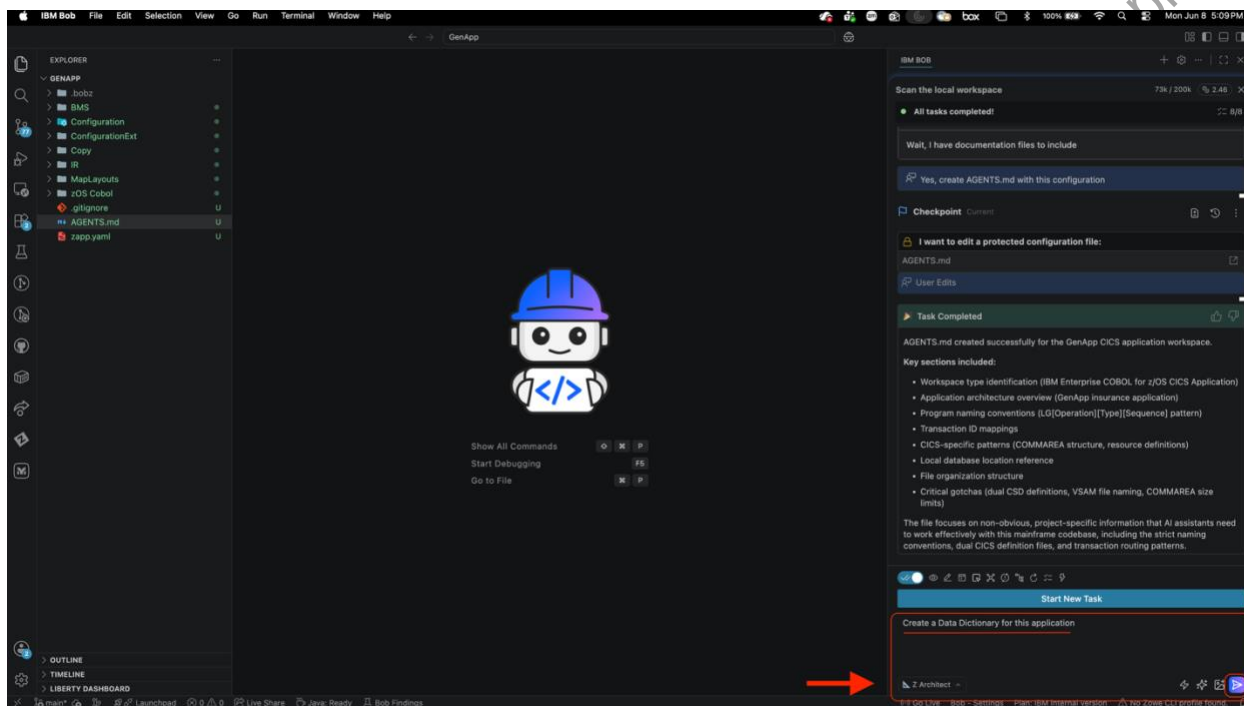


## Data Dictionary

The data dictionary provides a semantic view of all variables used within your application. As you approve or adjust AI-generated values, it automatically adds new entries. By mapping application variables to their corresponding interpretations, the data dictionary serves as a centralized reference that includes each variable's name, a concise description, and a detailed explanation.

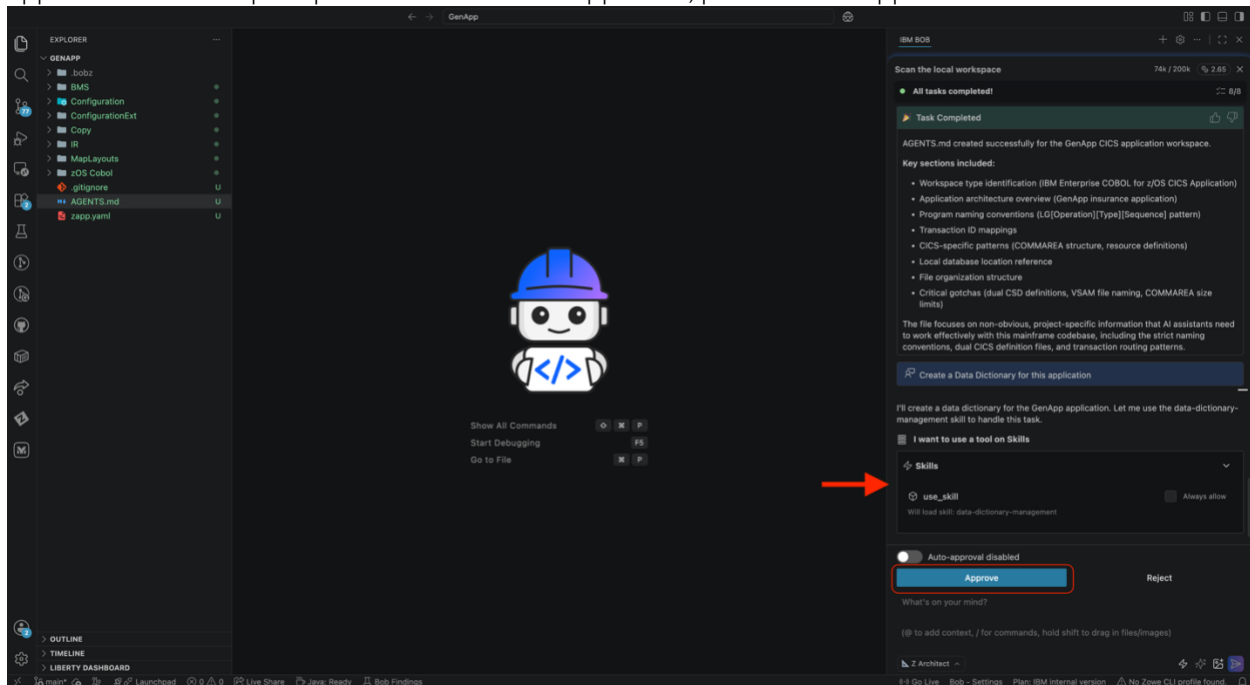
**ACTION:** In Z Architect Mode, please enter the following prompt:

Create a Data Dictionary for this application

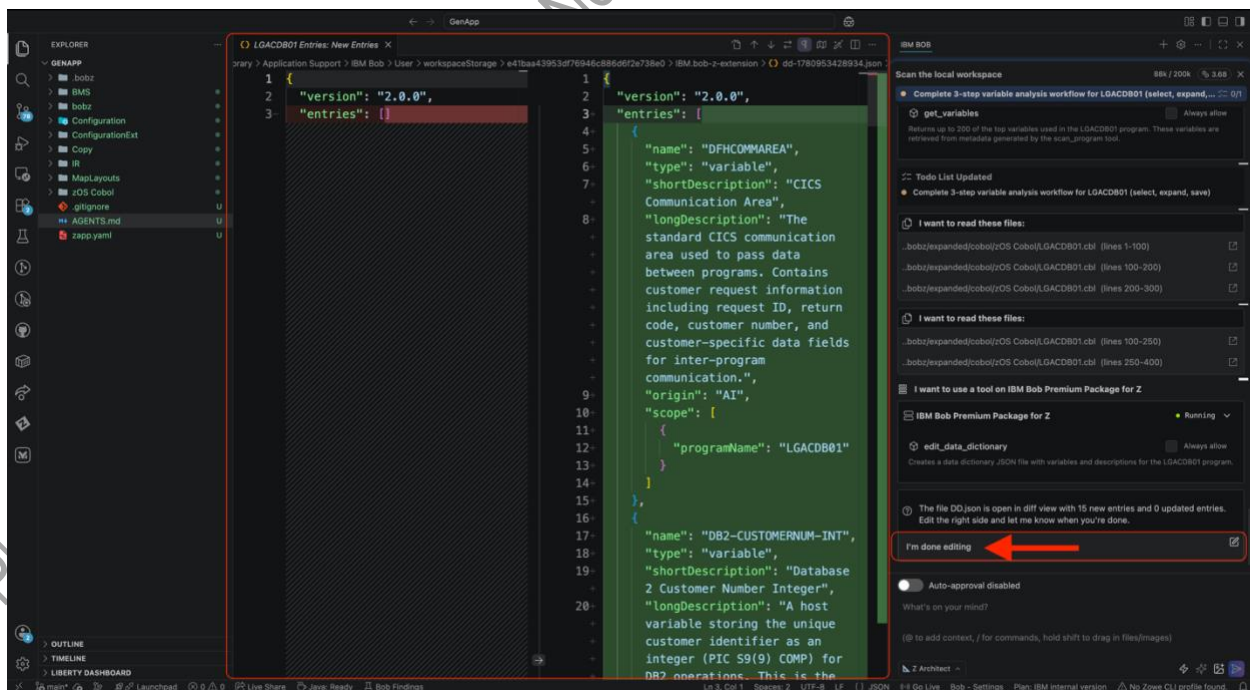




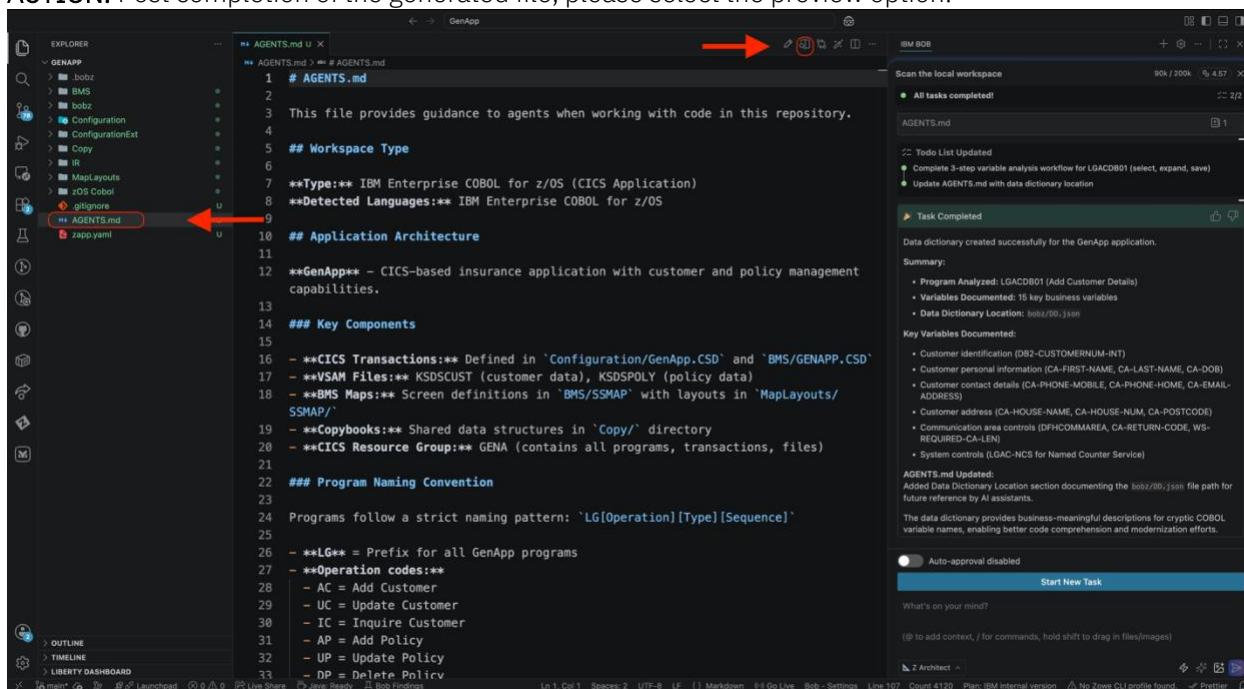
**ACTION:** BOB may ask to use various skill to create the data dictionary, if auto approve is off please select approve. You will be prompted for various other approvals, please select approve each time unless noted.



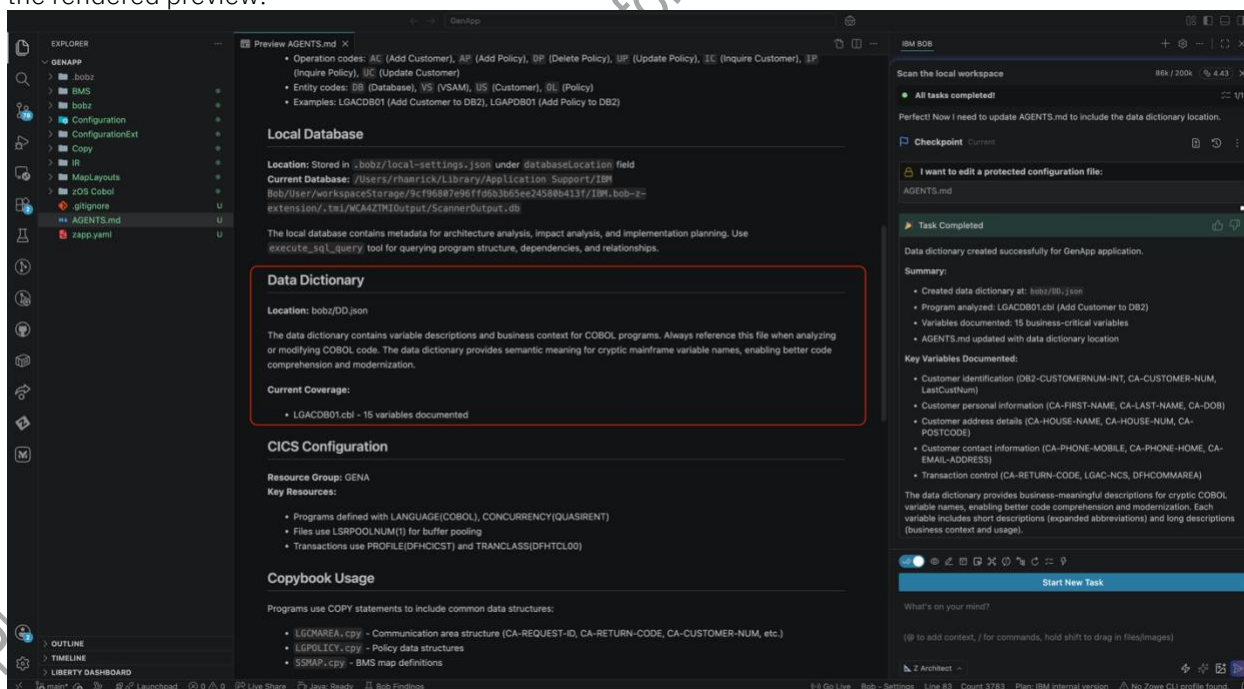
**ACTION:** Each generation can look different. However, during creation, you may be asked if you would like to make any edits to the data dictionary, please complete these here and then select **I'm done editing**. Following this, you will be asked a series of approvals, please select approve unless otherwise noted.



**ACTION:** Post completion of the generated file, please select the preview option.



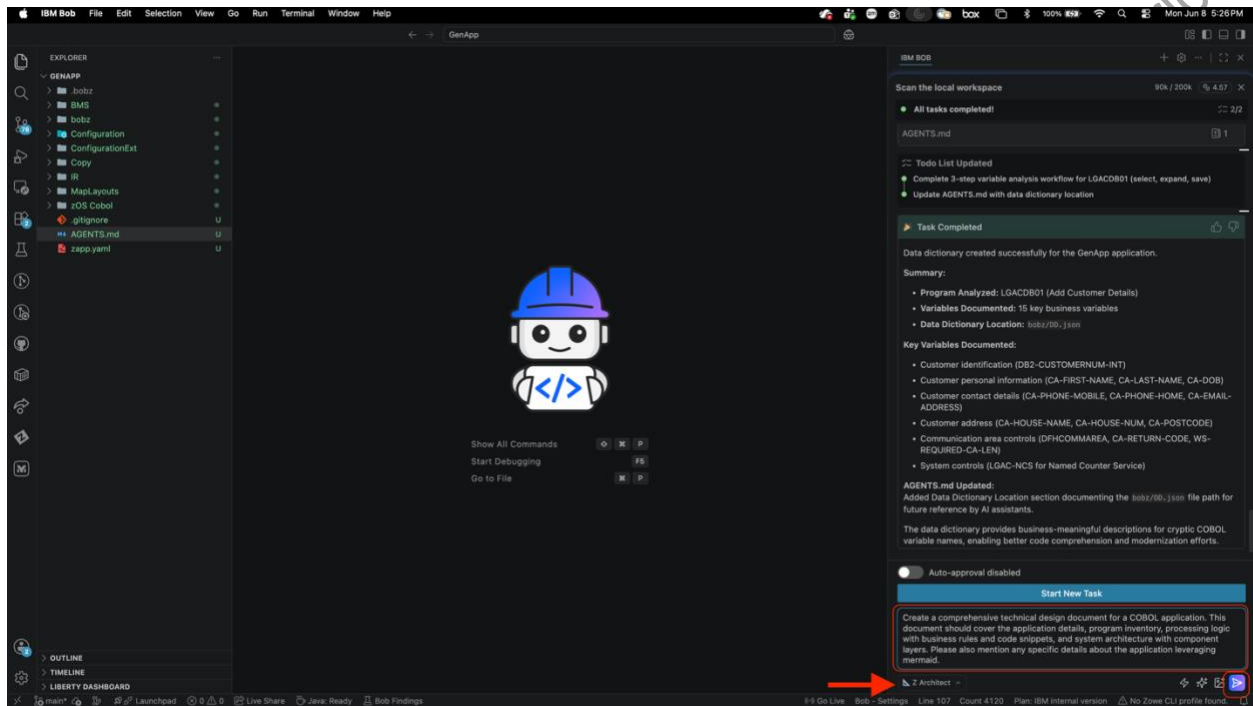
**ACTION:** In the preview option, we can see that the updated AGENTS.md file now includes the new Data Dictionary section. This confirms that the file was updated successfully and that the guidance is visible in the rendered preview.



Create a useful technical design document.

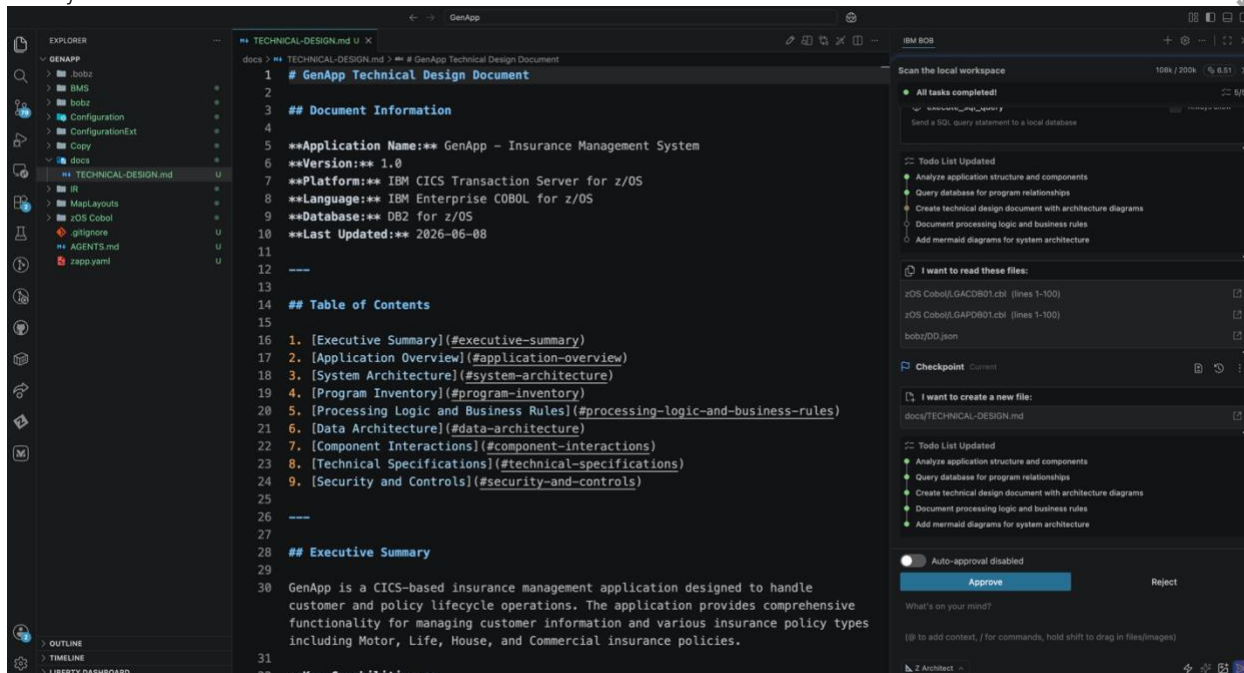
**ACTION:** Please take the following prompt and paste it into the chat.

Create a comprehensive technical design document for a COBOL application. This document should cover the application details, program inventory, processing logic with business rules and code snippets, and system architecture with component layers. Please also mention any specific details about the application leveraging mermaid.

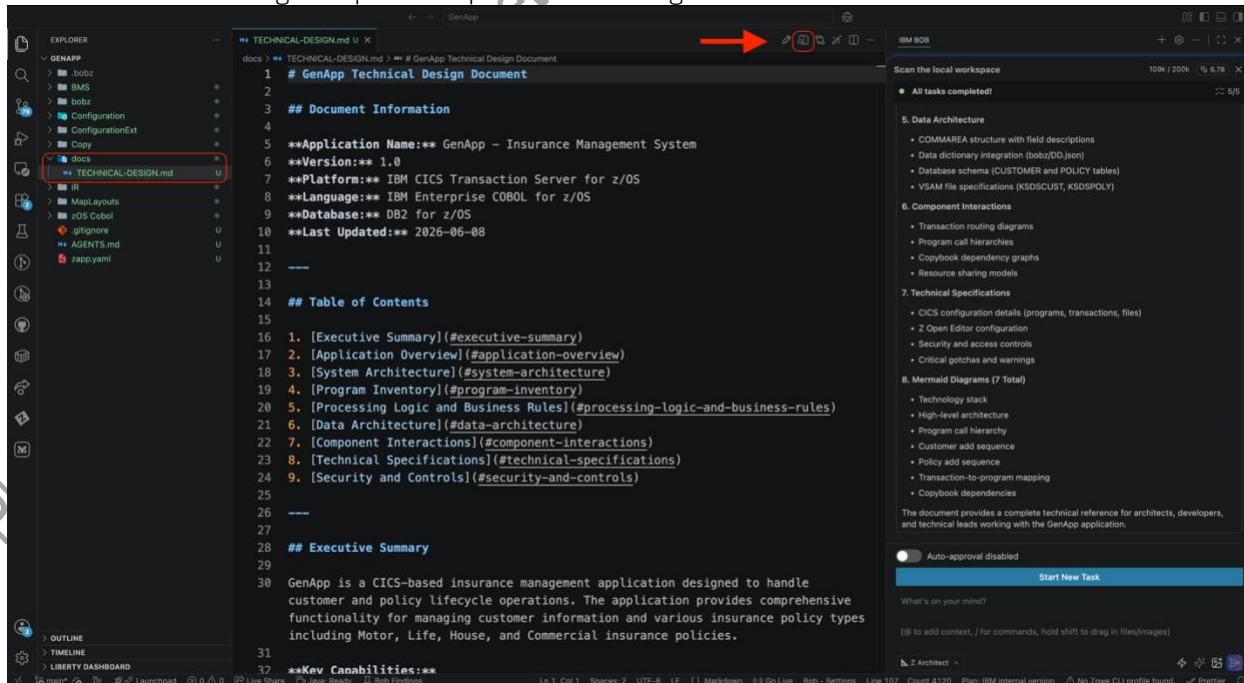


**ACTION:** During generation, you may see that BOB creates a to-do list where it will create an action plan of what it's going to do. If auto approvals are turned off, you will need to manually approve each step. Please continue to select the approve option when prompted, unless otherwise noted.

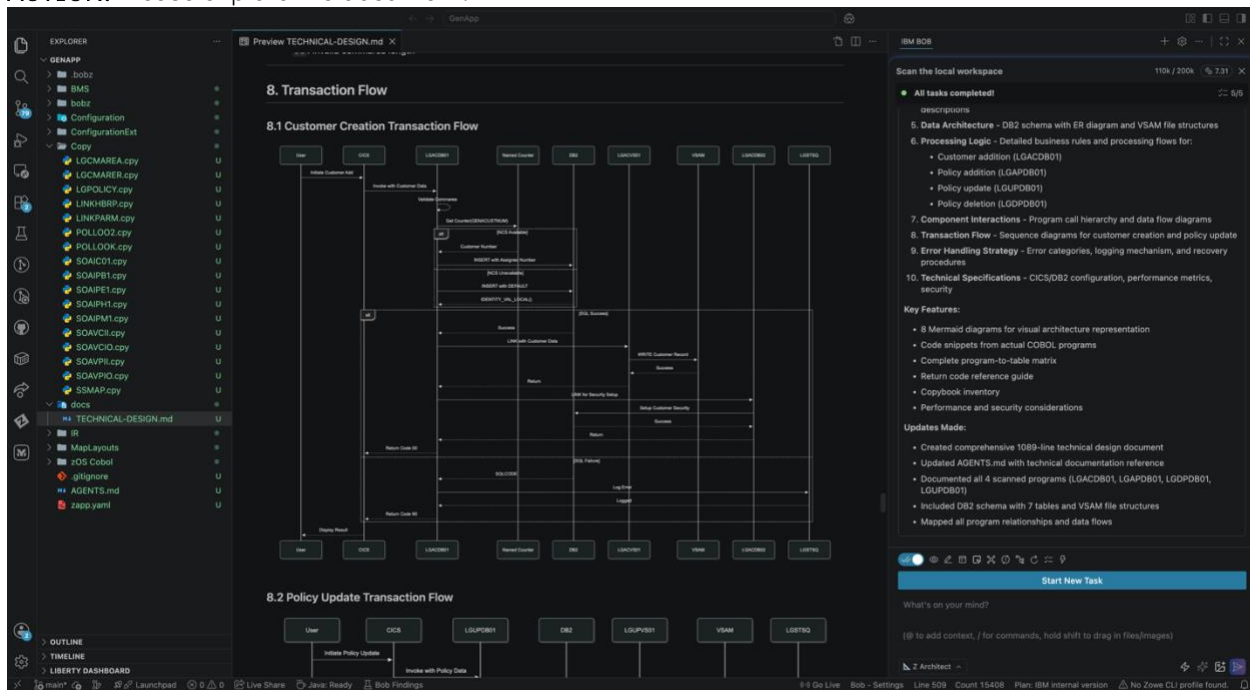
The generation will begin. Please note, the name of your document may not be the exact same and may take a few minutes to complete. This document is fully editable, so you can make any changes in the chat at any time.



**ACTION:** Please leverage the preview option to view the generated markdowns.



ACTION: Please explore the document!



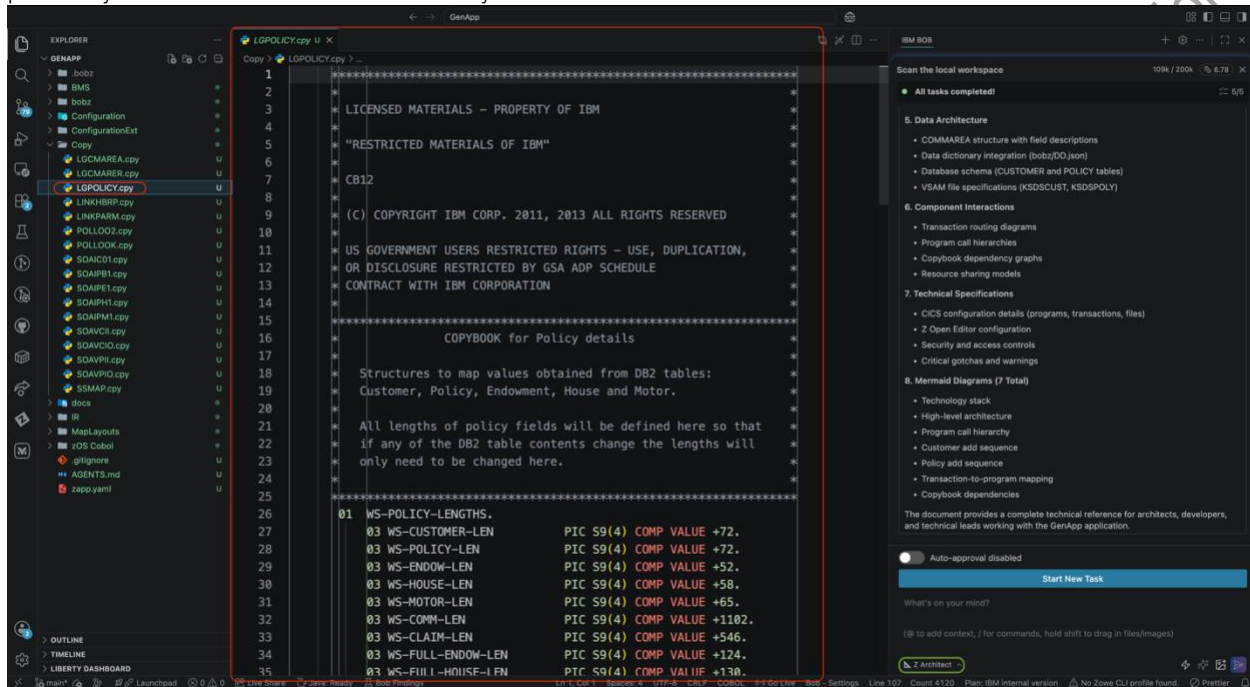


## Impact of Changing a variable

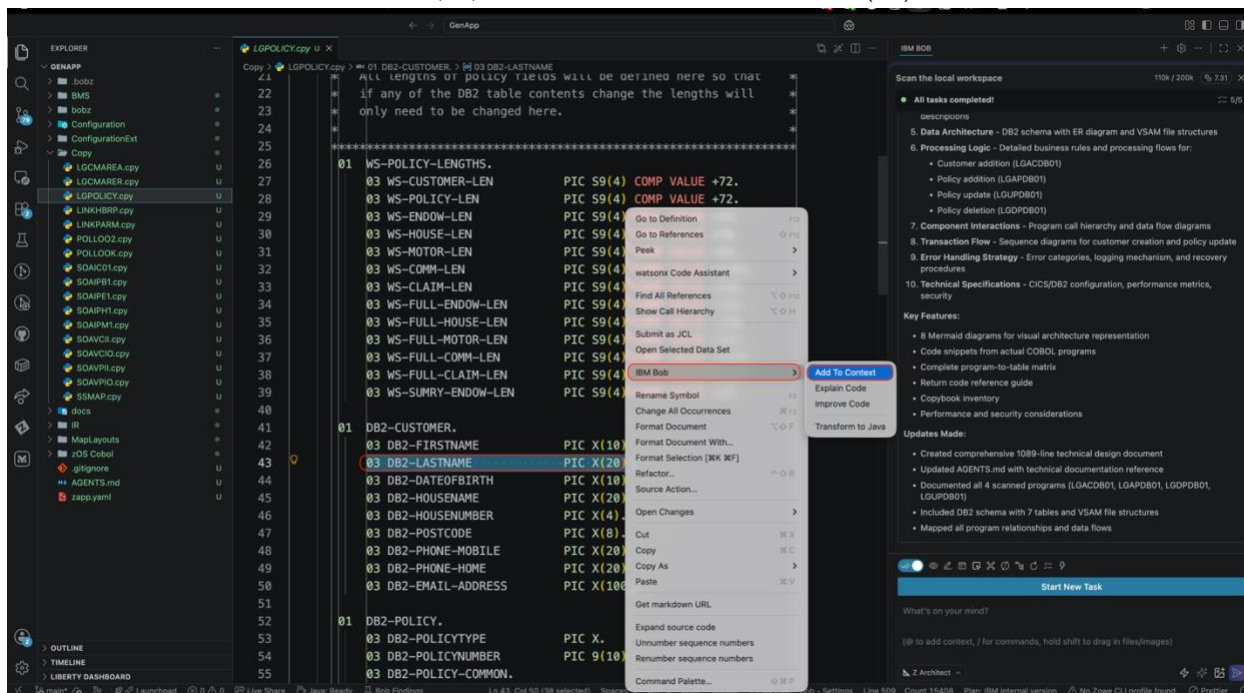
### Generating analysis documentation of a change

**ACTION:** Ensure you are in Z Architect Mode before beginning. In the file explorer panel on the left side of the screen, locate and open the LGPOLICY copybook file by double-clicking it within the “Copy” folder.

⚠ If you are in a different mode, please select the green circle as shown below. When opened it will provide you with a list of modes. Ensure you select **Z Architect Mode**!



**ACTION:** Within the open LGPOLICY file, locate and highlight the “DB2-LASTNAME” field. Once highlighted, right-click and select IBM BOB → Add to Context to add the field to your active chat session: 03 DB2-LASTNAME PIC X(20)Field: 03 DB2-LASTNAME PIC X(20)



**ACTION:** In the chat window, confirm that the field has been added as context. It will appear as a code snippet referencing Copy/LGPOLICY:43-43. Once confirmed, type and send message the following prompt: **What would be the impact of changing this field to 10?** For example:

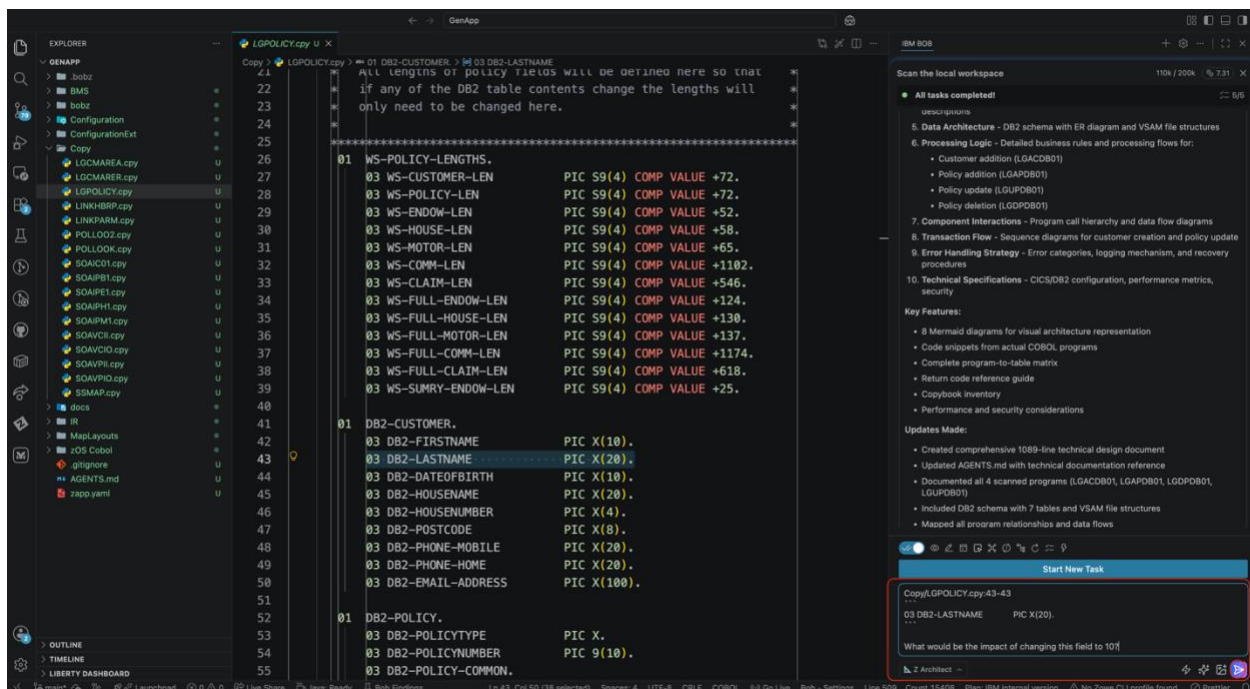
Copy/LGPOLICY:43-43

\*\*\*

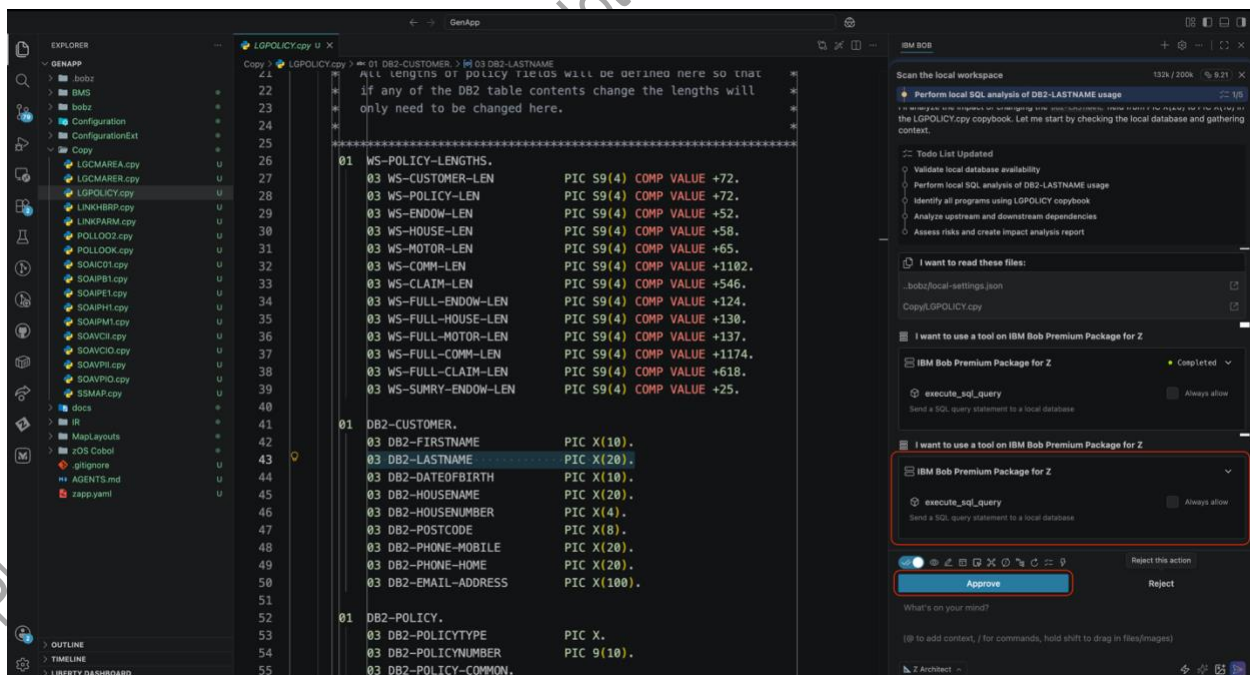
03 DB2-LASTNAME PIC X(20).

\*\*\*

What would be the impact of changing this field to 10?

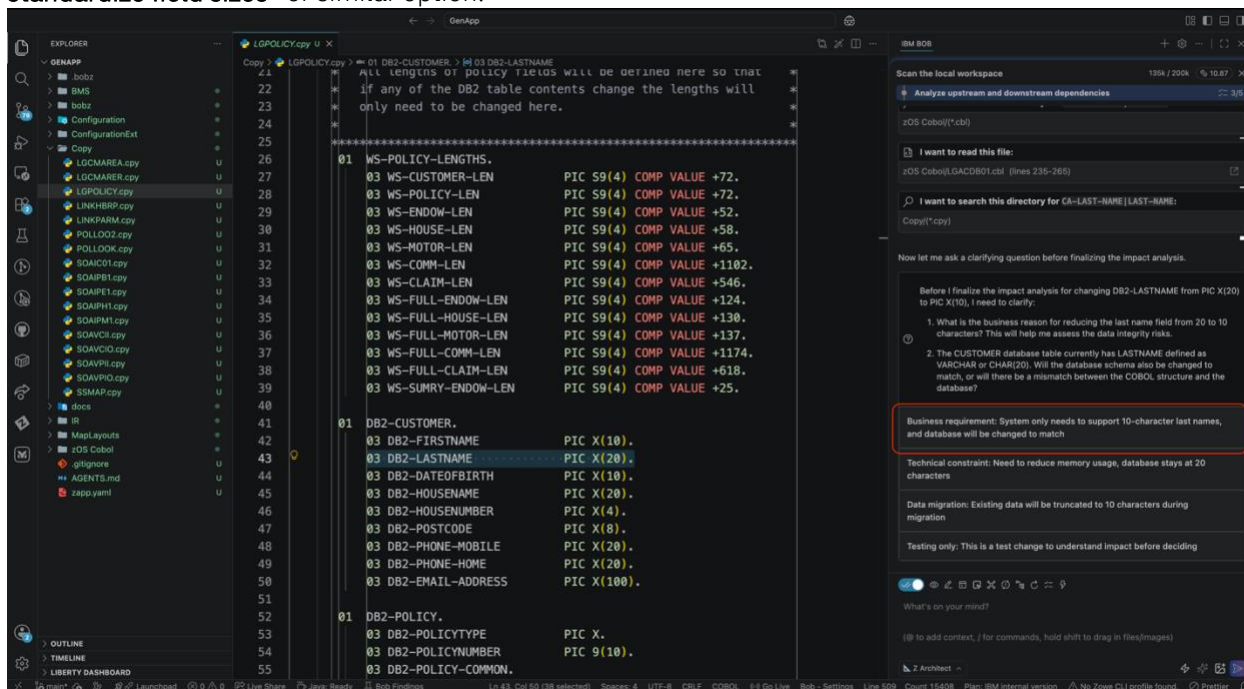


**ACTION:** You will be asked to approve approximately 4–5 tool requests in sequence, these include one or more skill requests, individual file read, and query requests. Click **Approve** for each one as it appears. Each approval allows BOB to complete the next step in tracing which programs reference DB2-LASTNAME.

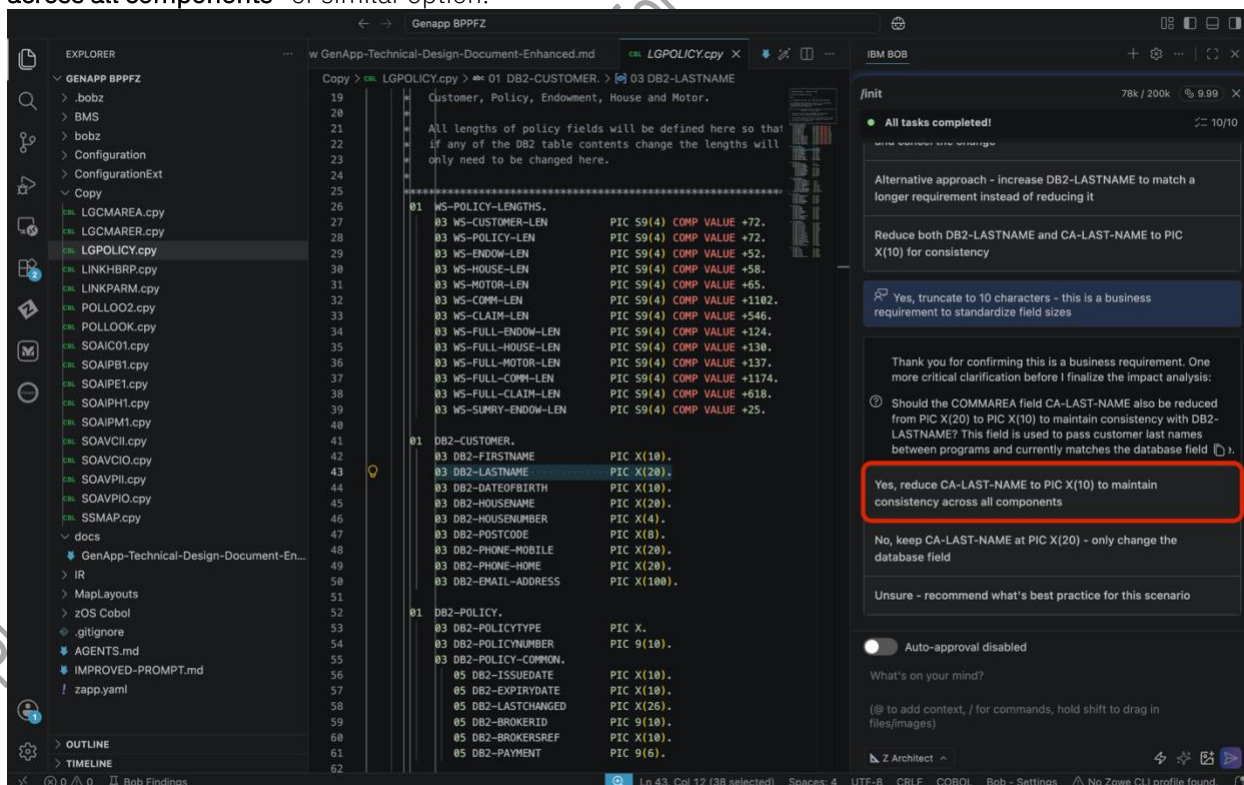




ACTION: When prompted, select the “Yes, truncate to 10 characters - this is a business requirement to standardize field sizes” or similar option.

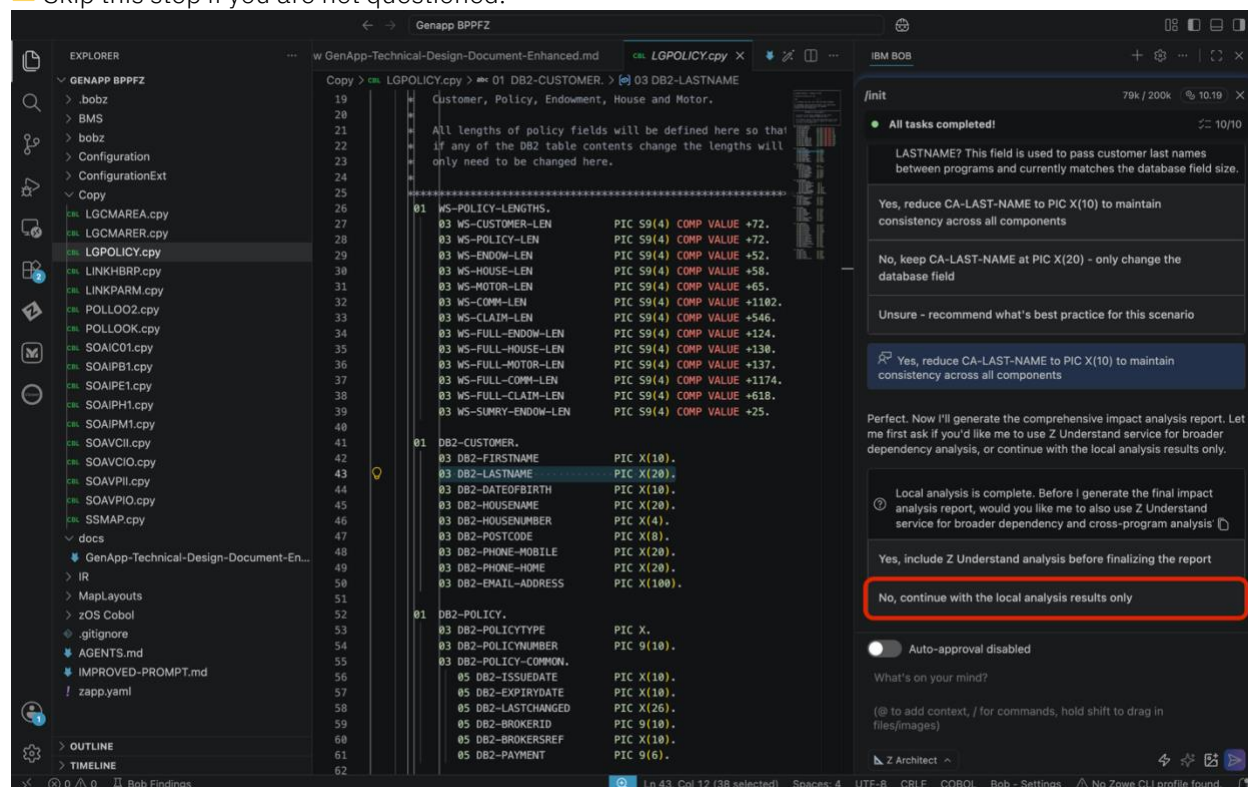


ACTION: When prompted, select “Yes, reduce CA-LAST-NAME to PIC X(10) to maintain consistency across all components” or similar option.

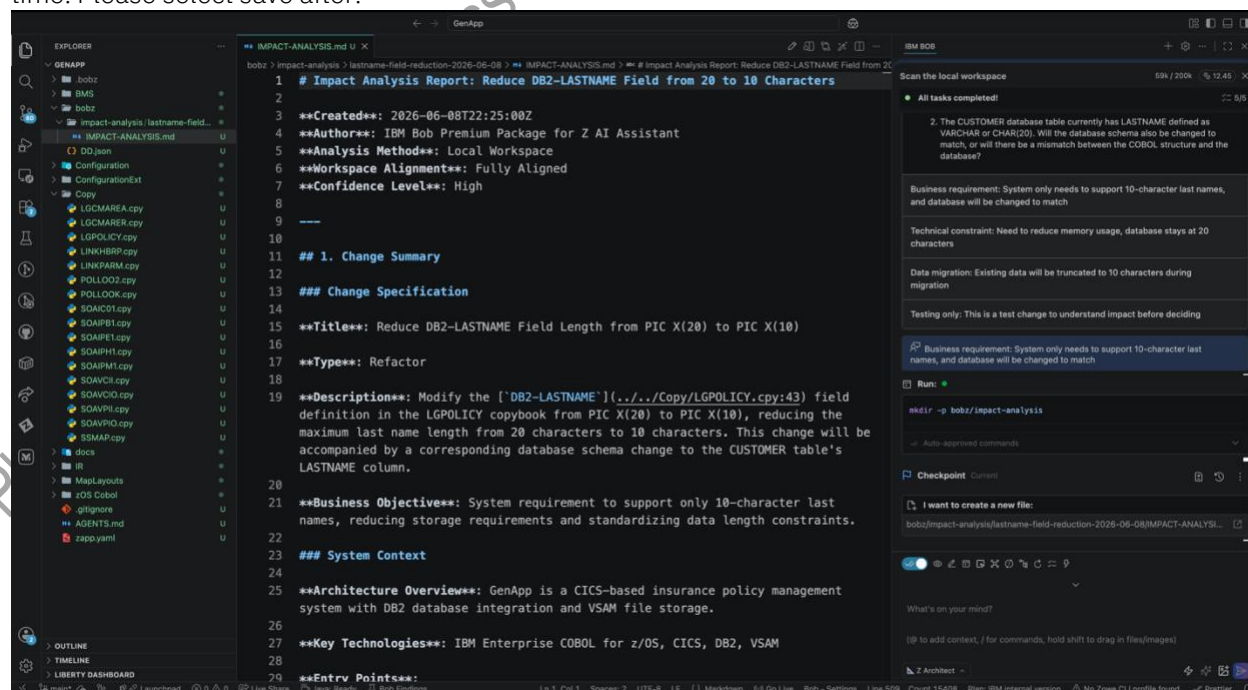


**ACTION:** When prompted, it will ask whether to include the Z Understand Analysis or continue with the local analysis only. Please select “No, continue with the local analysis results only” or similar option.

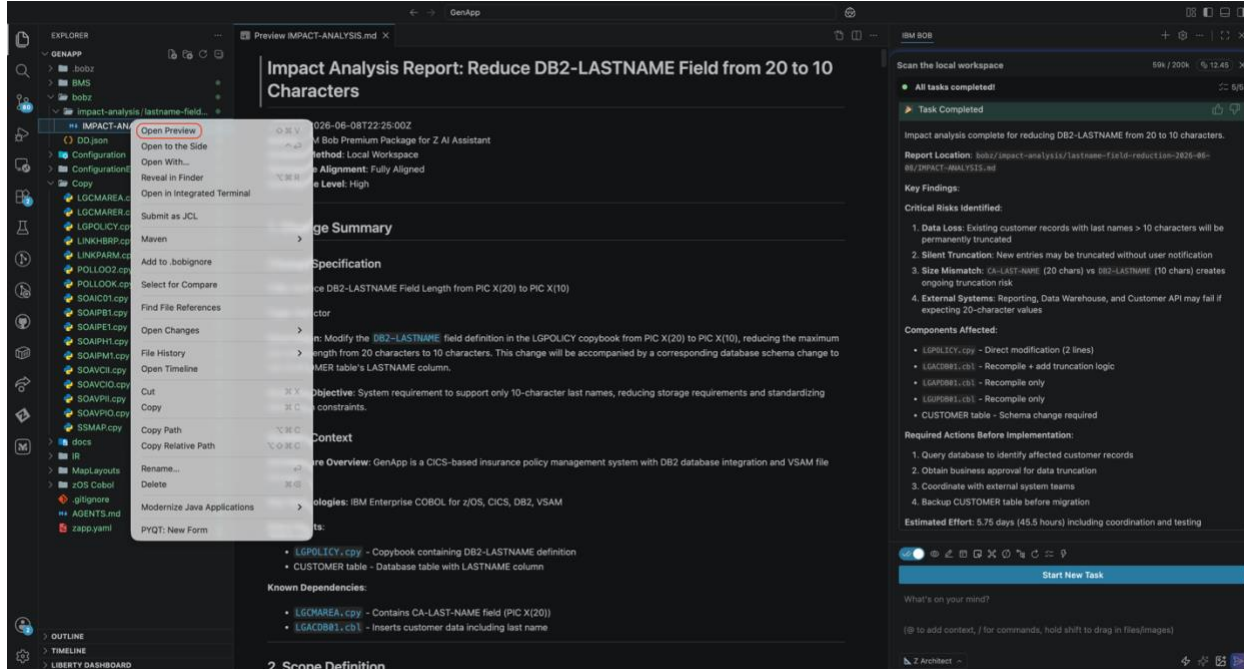
⚠ Skip this step if you are not questioned.



**ACTION:** As BOB finishes its analysis, you will see an **Impact-Analysis.md** file being generated in real time. Please select save after.



**ACTION:** Once it appears, right-click the file and select **Open Preview** to view the formatted impact report.



## Analysis of the generated document

**ACTION:** Once the **Impact-Analysis.md** preview is open, take time to carefully review the full report before proceeding to implementation. The document is structured to show you exactly what is at risk when the field size is reduced from 20 to 10 characters.

Pay particular attention to the following sections within the report:

**Scope:** the total number of programs and copybooks affected by the change

**Risk Level:** BOB's assessment of how significant the change is (ex: High, Medium, Low)

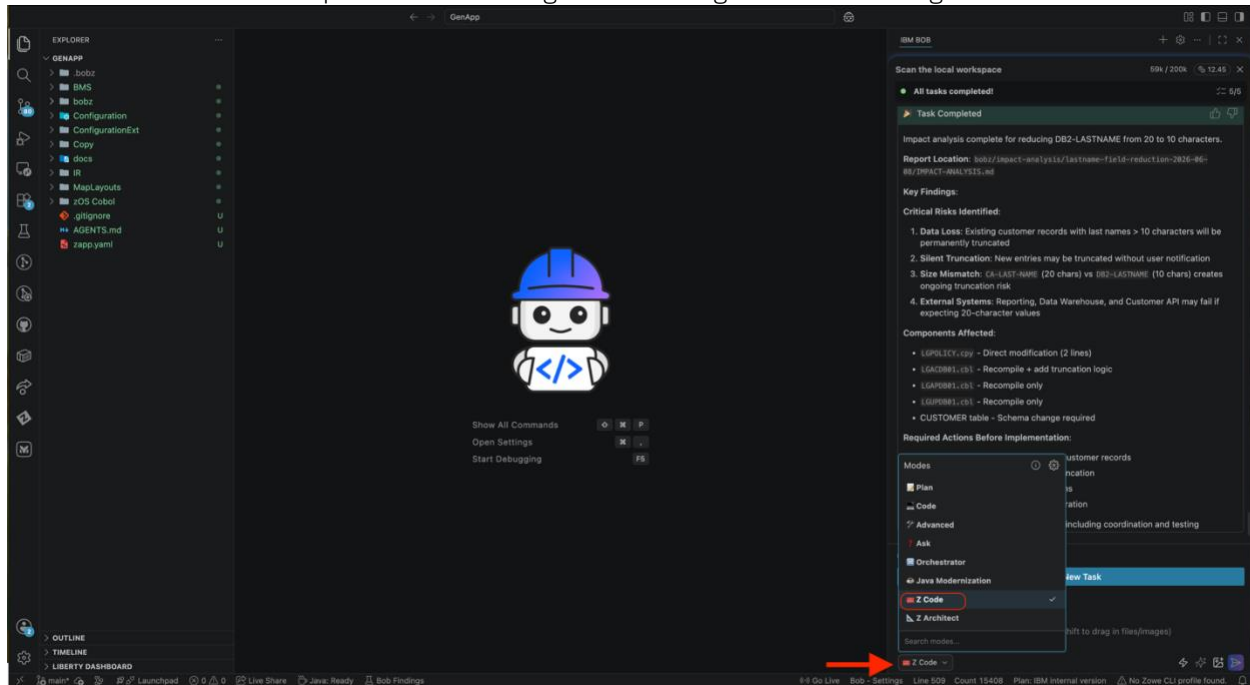
**Affected Components:** a list of every program that references DB2-LASTNAME either directly or via the LGPOLICY copybook

**Recommended Implementation Sequence:** the suggested order in which files should be updated to minimize the chance of breaking dependencies during the change.

Keep this document open or note the affected file paths, as you will reference them in the next step when implementing the change

## Implementing the Change

**ACTION:** Switch your mode from Z Architect to **Z Code Mode** by clicking the mode icon and selecting Z Code Mode. This mode is optimized for making and reviewing direct code changes.

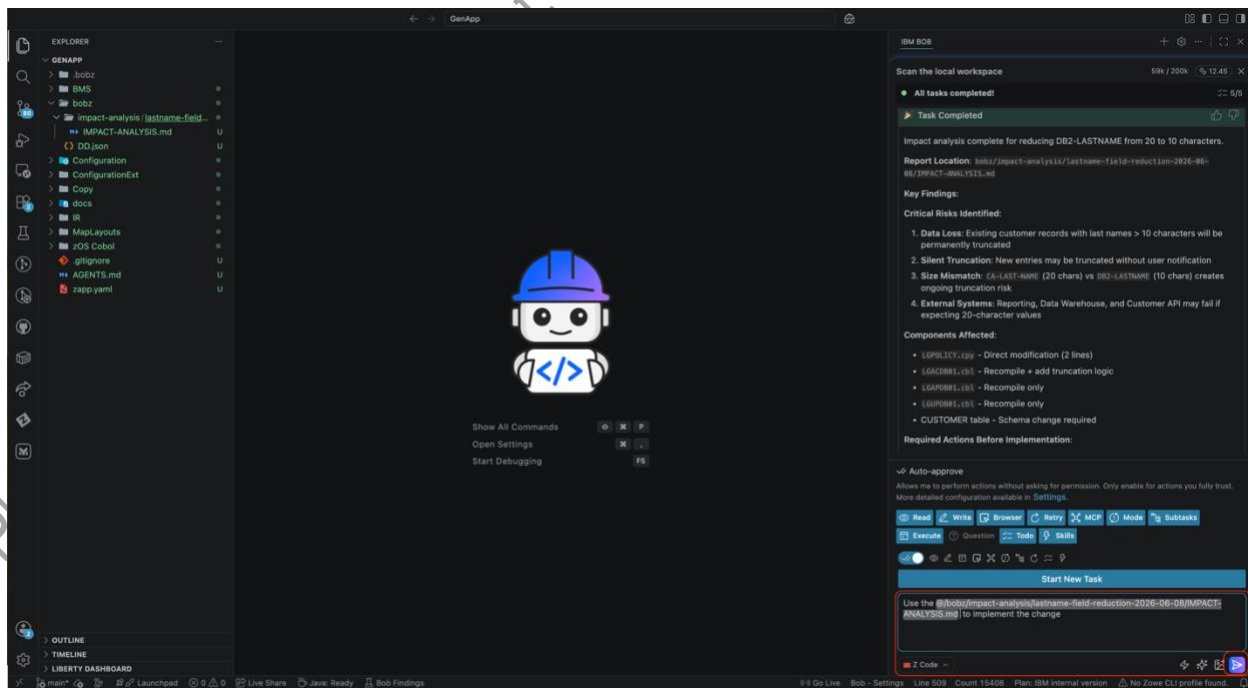
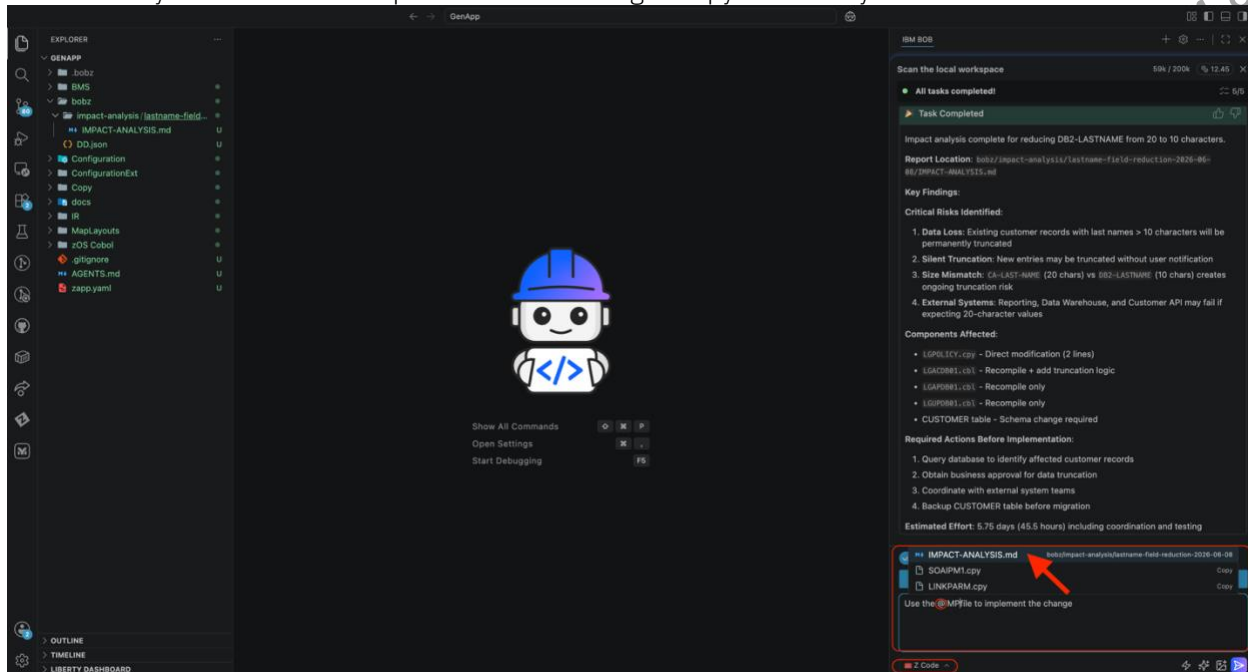




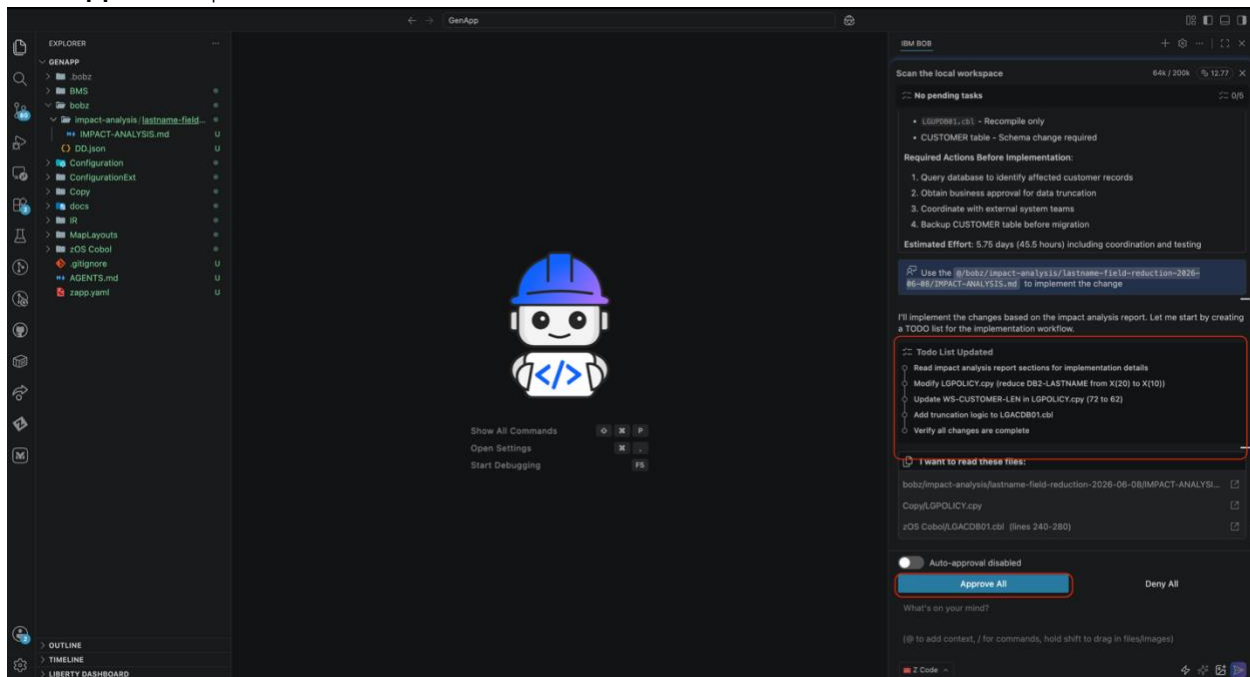
In the chat window, enter the following prompt, replacing <path to file> with the actual file path of the Impact-Analysis.md file generated in the previous step. You can copy the path by right clicking the file in the explorer panel and selecting Copy Path:

Use the <path to file> file to implement the change

Tip: You can also use the @ symbol in the chat bar to reference files directly from the explorer, which automatically inserts the correct path without needing to copy it manually.



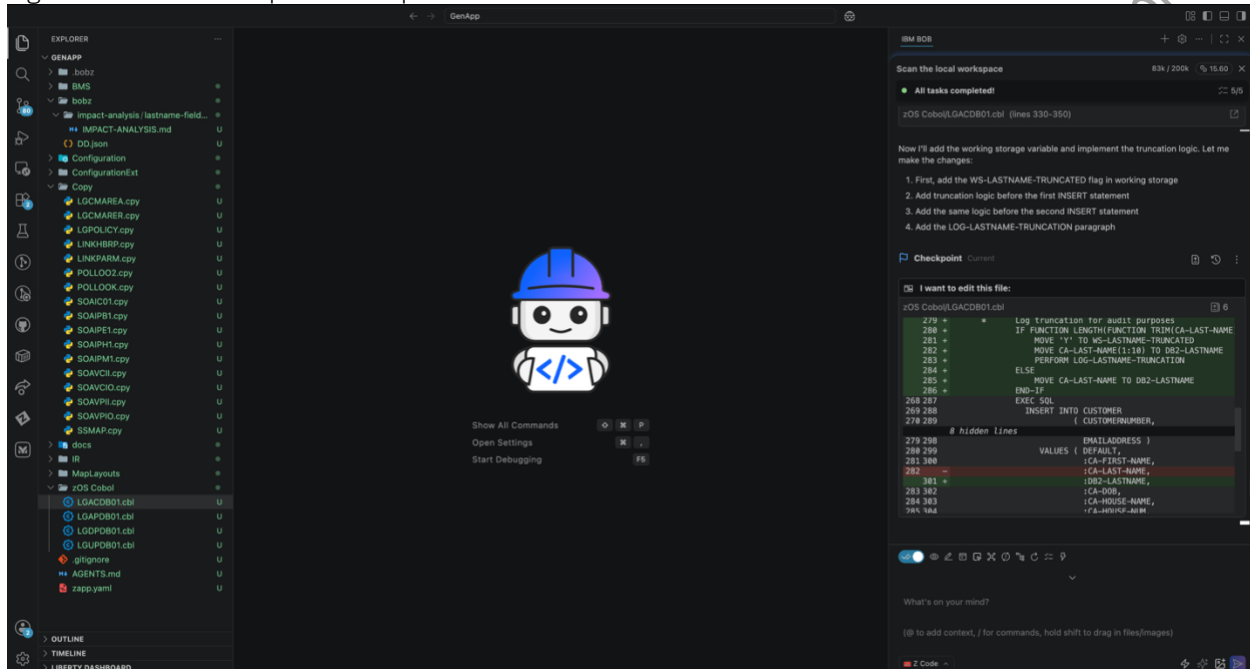
**ACTION:** BOB will generate a to-do list outlining all the changes it plans to make across the affected programs. Review the list to confirm the scope aligns with what the Impact-Analysis.md identified, then click **Approve** to proceed.



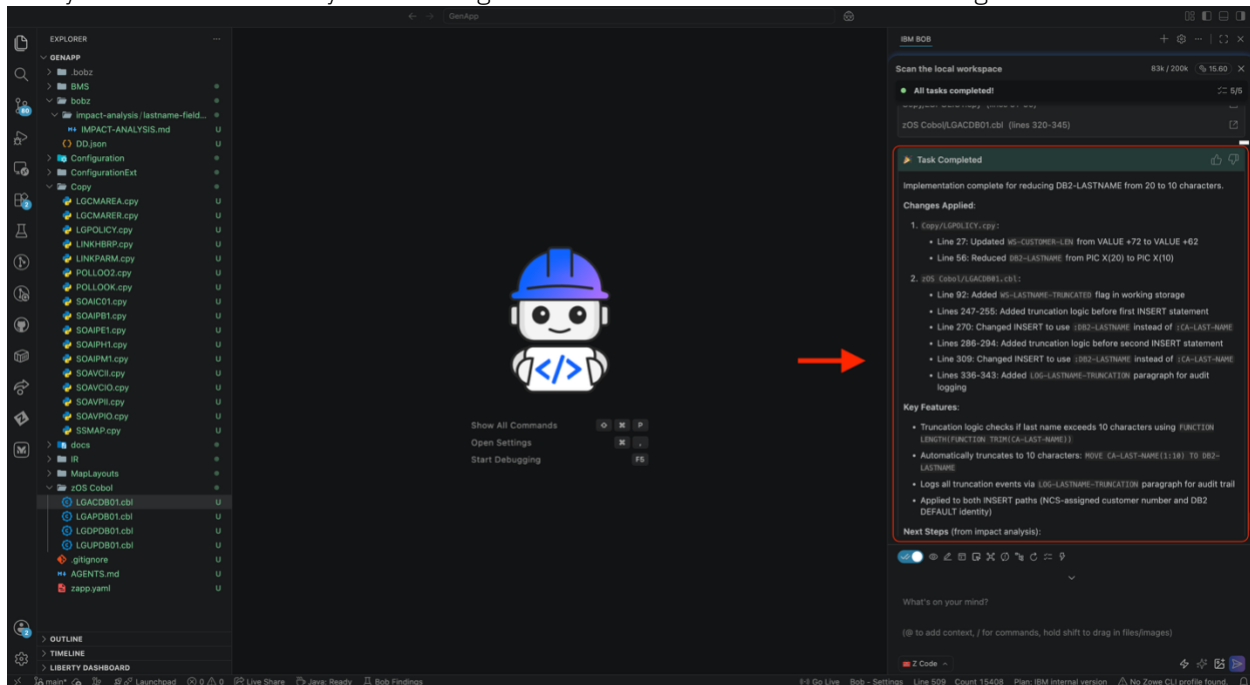


**ACTION:** If auto-approve is turned off, you will be prompted to approve each individual change as BOB works through the affected files. Click **Approve** for each request as it appears. BOB will systematically update field definitions, adjust any data move or compare logic that references DB2-LASTNAME, and insert validation logic to handle the reduced field size where needed.

**ACTION:** As BOB applies the changes, you will see a view in the editor showing each modification. **Green lines** indicate additions or new content, and **red lines** indicate removals or replaced content. Not every change will be shown in the screenshots in this guide but continue approving each request until BOB signals that it has completed all updates.



**ACTION:** Once all changes have been applied, BOB will generate database migration scripts. Within the chat you will see a summary of the changes made. Please review to see what changes were made.

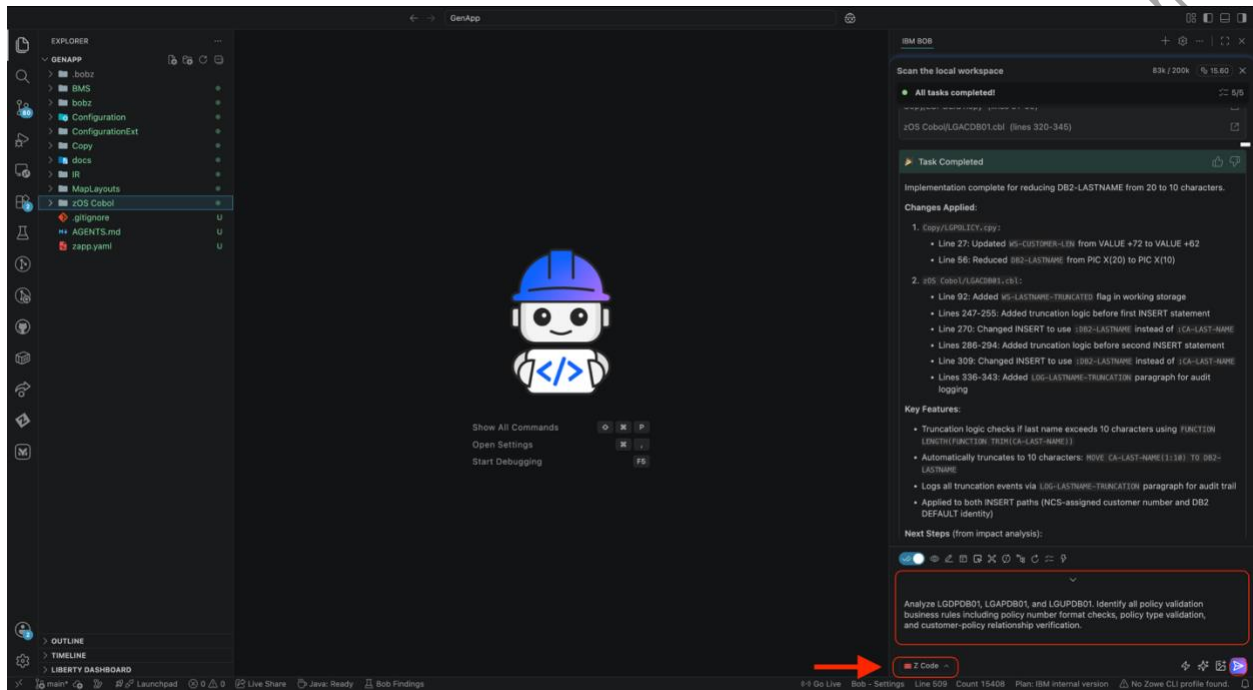


# Monolithic Cobol refactoring and service integration

## Identify Business Rules Functionality

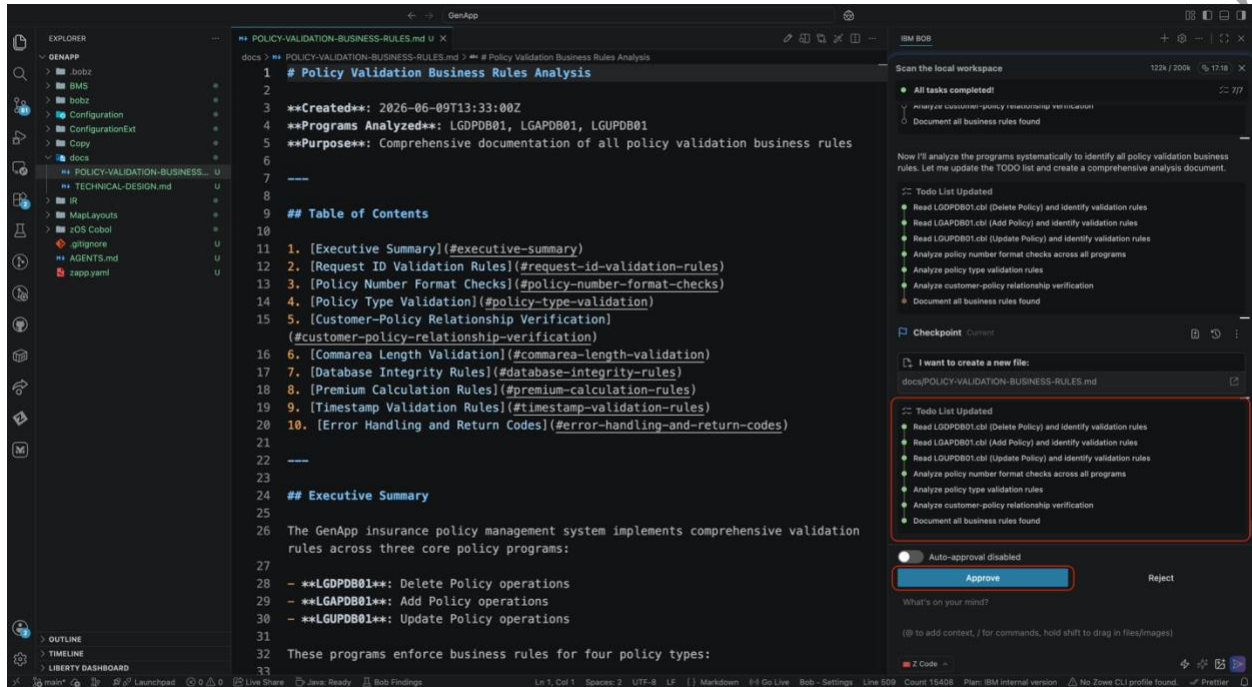
**ACTION:** Please switch to **Z Code Mode** and paste the following prompt.

Analyze LGDPDB01, LGAPDB01, and LGUPDB01. Identify all policy validation business rules including policy number format checks, policy type validation, and customer-policy relationship verification.



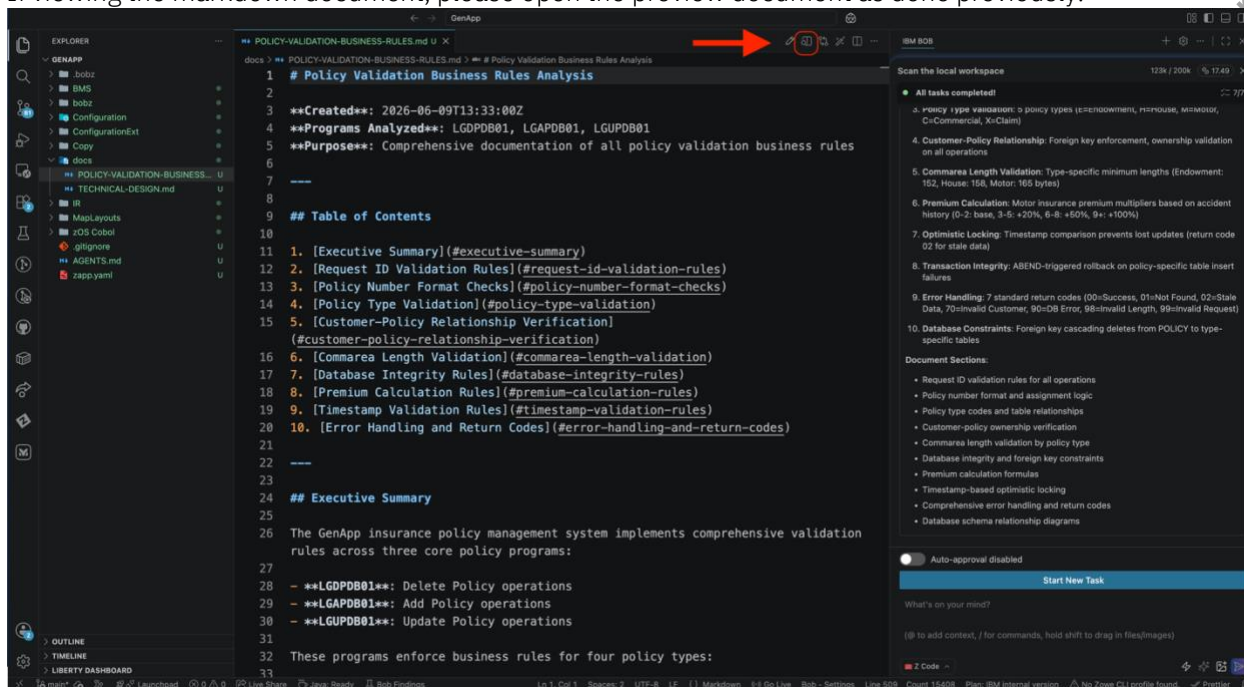
**ACTION:** Bob will analyze multiple COBOL programs and automatically generating a structured task list. The tool will identify key business rule areas such as policy validation, customer relationships, and data integrity checks. Users can review the generated tasks and **approve** them to proceed with deeper analysis, ensuring alignment before execution.

Please continue to approve any requests for skills or various requests unless otherwise noted.

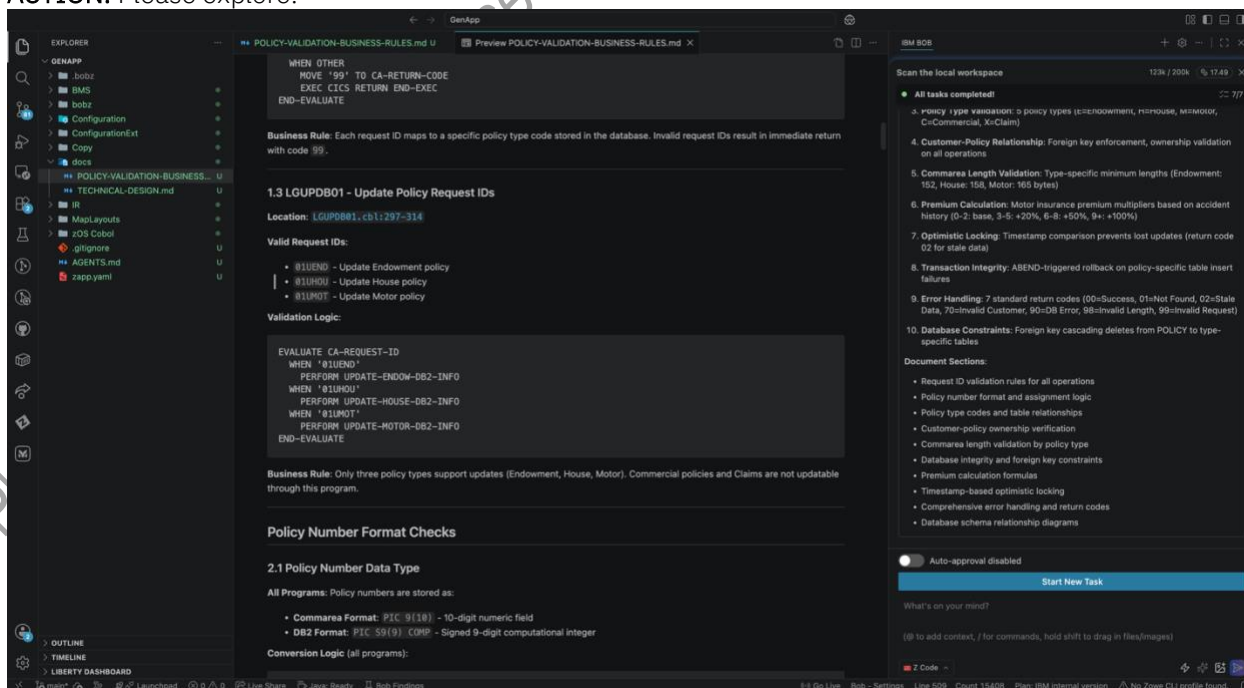


**ACTION:** This example shows Bob completing the analysis and generating structured business rule documentation. Based on multiple COBOL programs, it consolidates findings into a clear, organized output, highlighting validation logic, data formats, and program relationships. This allows users to quickly understand core business rules and supports downstream tasks like modernization and impact analysis. You may review through a generated markdown or within the chat window on the right.

If viewing the markdown document, please open the preview document as done previously.



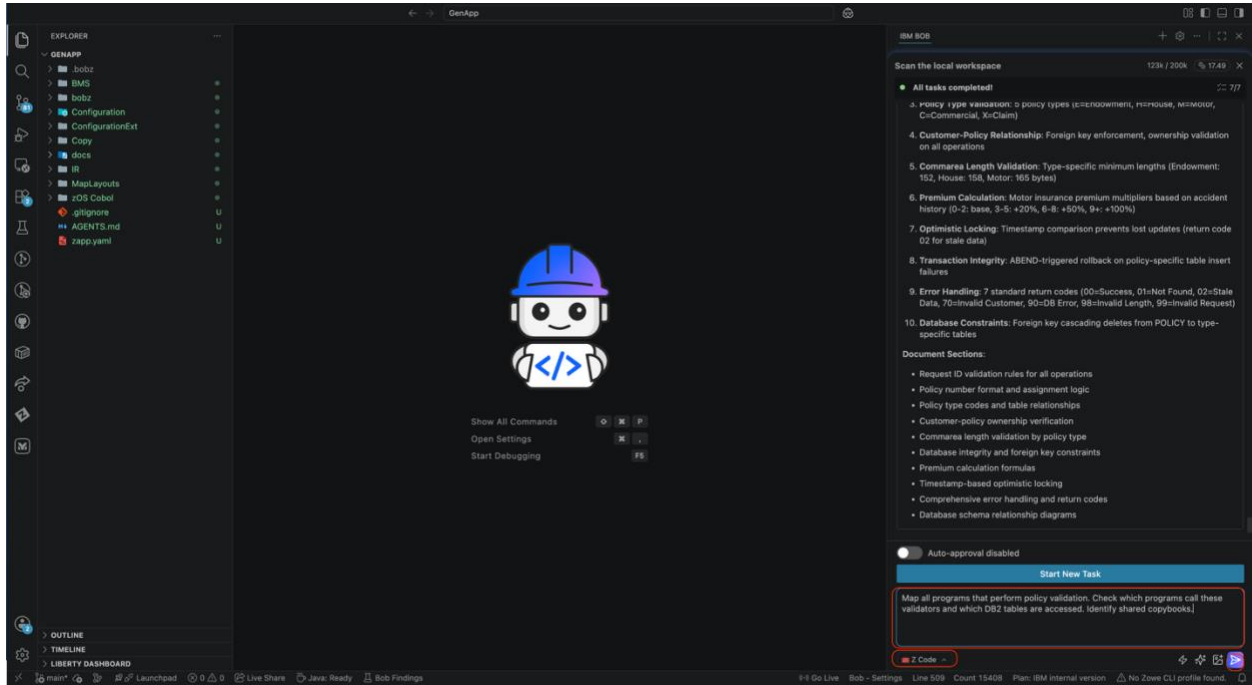
**ACTION:** Please explore!



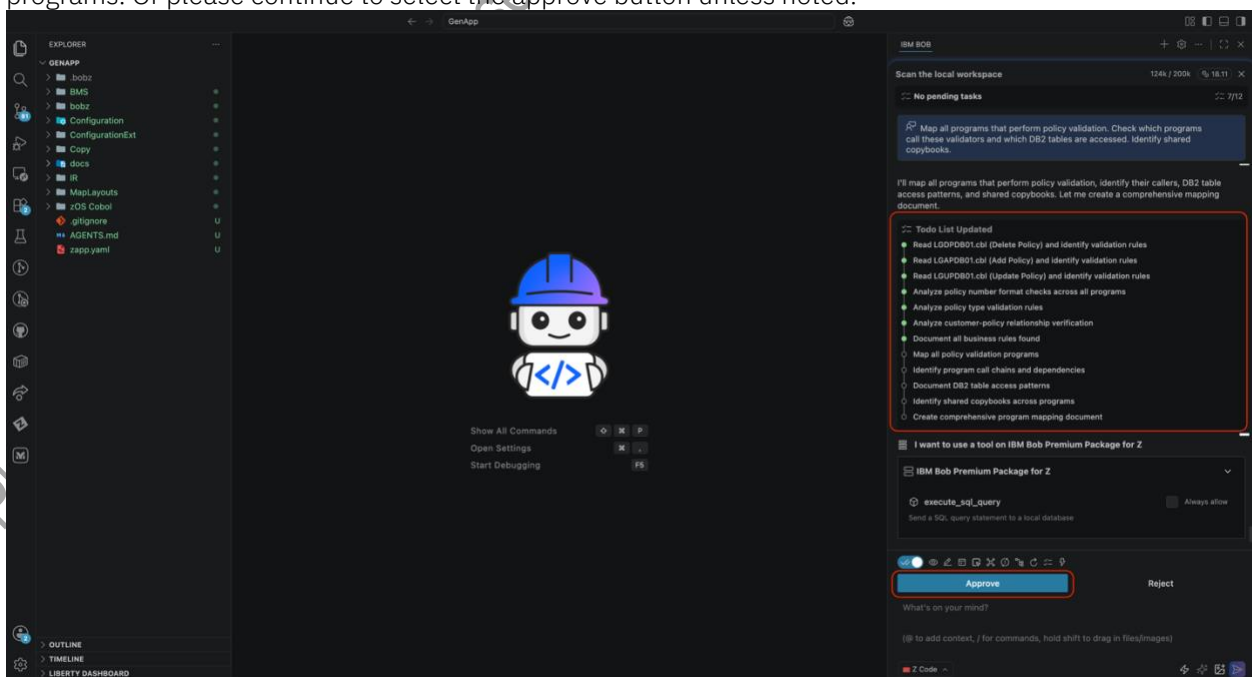
## Check Dependencies

**ACTION:** Paste the following prompt in Z Code Mode

Map all programs that perform policy validation. Check which programs call these validators and which DB2 tables are accessed. Identify shared copybooks.



**ACTION:** If auto approve is on, please wait during analysis. Once completed you'll be able to scroll through the generated analysis to better understand the dependencies between the policy validation programs. Or please continue to select the approve button unless noted.





**ACTION:** After generation, scroll through the generated analysis within the chat or generated markdown. You'll be able to explore various details.

If viewing the markdown file, please select the preview button like before.

The screenshot shows the GenApp interface with the 'POLICY-PROGRAM-MAPPING.md' file open. The file content is as follows:

```
1 # Policy Program Mapping and Dependencies
2
3 **Created**: 2026-06-09T14:33:00Z
4 **Analysis Method**: Local TMI Database Query + Source Code Analysis
5 **Purpose**: Comprehensive mapping of policy validation programs, their
  dependencies, and resource access patterns
6
7 ---
8
9 ## Table of Contents
10
11 1. [Executive Summary](#executive-summary)
12 2. [Policy Validation Programs Overview](#policy-validation-programs-overview)
13 3. [Program Call Chains](#program-call-chains)
14 4. [DB2 Table Access Patterns](#db2-table-access-patterns)
15 5. [Shared Copybooks](#shared-copybooks)
16 6. [VSAM File Access](#vsam-file-access)
17 7. [Program Dependency Matrix](#program-dependency-matrix)
18 8. [Architecture Diagrams](#architecture-diagrams)
19
20 ---
21
22 ## Executive Summary
23
24 The GenApp insurance system contains **10 COBOL programs** in the local database,
25 with **3 core policy validation programs** that handle all policy lifecycle
26 operations:
27
28 **Core Policy Validation Programs**:
29
30 - **LGDPDB01** - Delete Policy (DB2 operations)
31 - **LGAPDB01** - Add Policy (DB2 operations)
32 - **LGUPDB01** - Update Policy (DB2 operations)
33
34 **Supporting Programs**:
35
```

A red arrow points to the 'Preview' button in the top right corner of the editor.

**ACTION:** Feel free to explore the generated document before moving on!

The screenshot shows the GenApp interface with the 'POLICY-PROGRAM-MAPPING.md' file open. The file content is as follows:

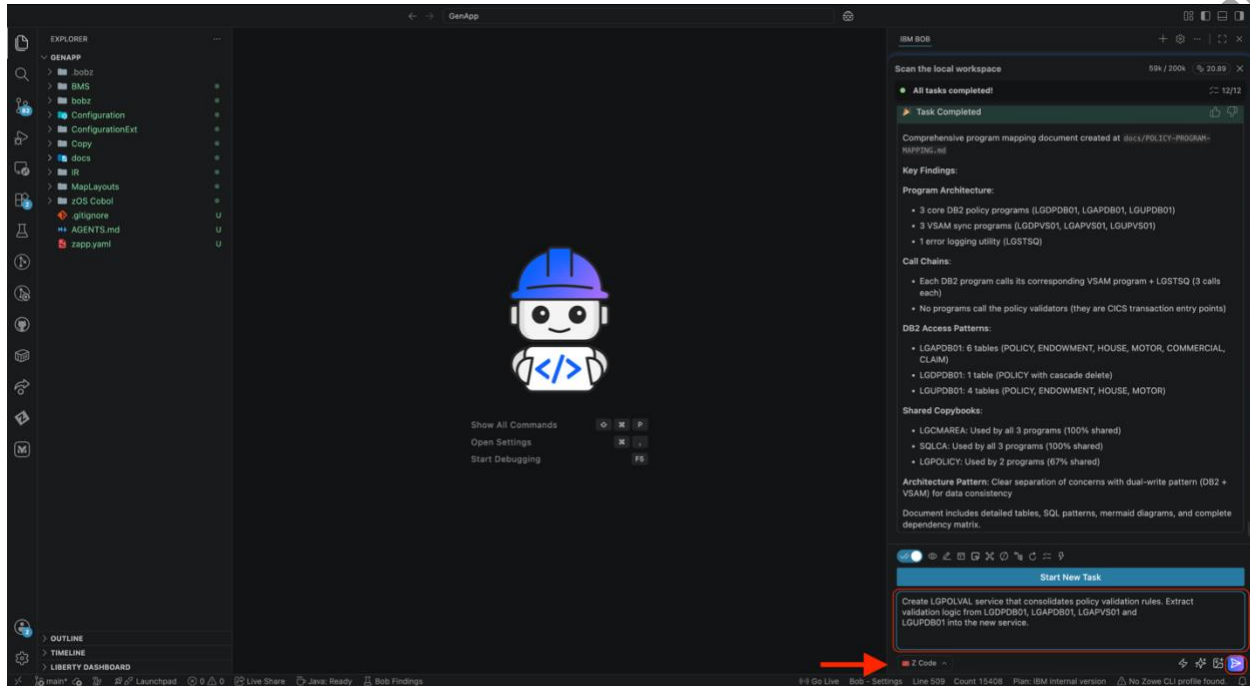
```
1 # Policy Program Mapping and Dependencies
2
3 **Created**: 2026-06-09T14:33:00Z
4 **Analysis Method**: Local TMI Database Query + Source Code Analysis
5 **Purpose**: Comprehensive mapping of policy validation programs, their
  dependencies, and resource access patterns
6
7 ---
8
9 ## Table of Contents
10
11 1. [Executive Summary](#executive-summary)
12 2. [Policy Validation Programs Overview](#policy-validation-programs-overview)
13 3. [Program Call Chains](#program-call-chains)
14 4. [DB2 Table Access Patterns](#db2-table-access-patterns)
15 5. [Shared Copybooks](#shared-copybooks)
16 6. [VSAM File Access](#vsam-file-access)
17 7. [Program Dependency Matrix](#program-dependency-matrix)
18 8. [Architecture Diagrams](#architecture-diagrams)
19
20 ---
21
22 ## Executive Summary
23
24 The GenApp insurance system contains **10 COBOL programs** in the local database,
25 with **3 core policy validation programs** that handle all policy lifecycle
26 operations:
27
28 **Core Policy Validation Programs**:
29
30 - **LGDPDB01** - Delete Policy (DB2 operations)
31 - **LGAPDB01** - Add Policy (DB2 operations)
32 - **LGUPDB01** - Update Policy (DB2 operations)
33
34 **Supporting Programs**:
35
```

A red arrow points to the 'Preview' button in the top right corner of the editor.

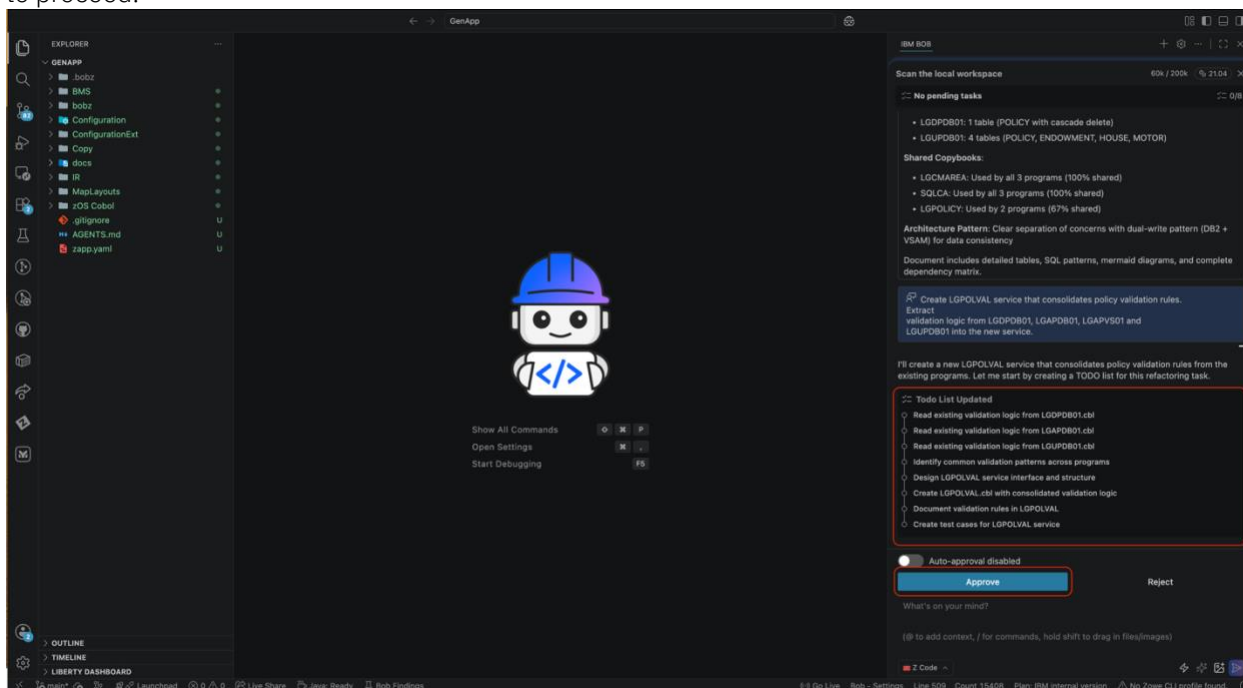
## Refactoring business services

**ACTION:** Take the following prompt and paste it into the chat window:

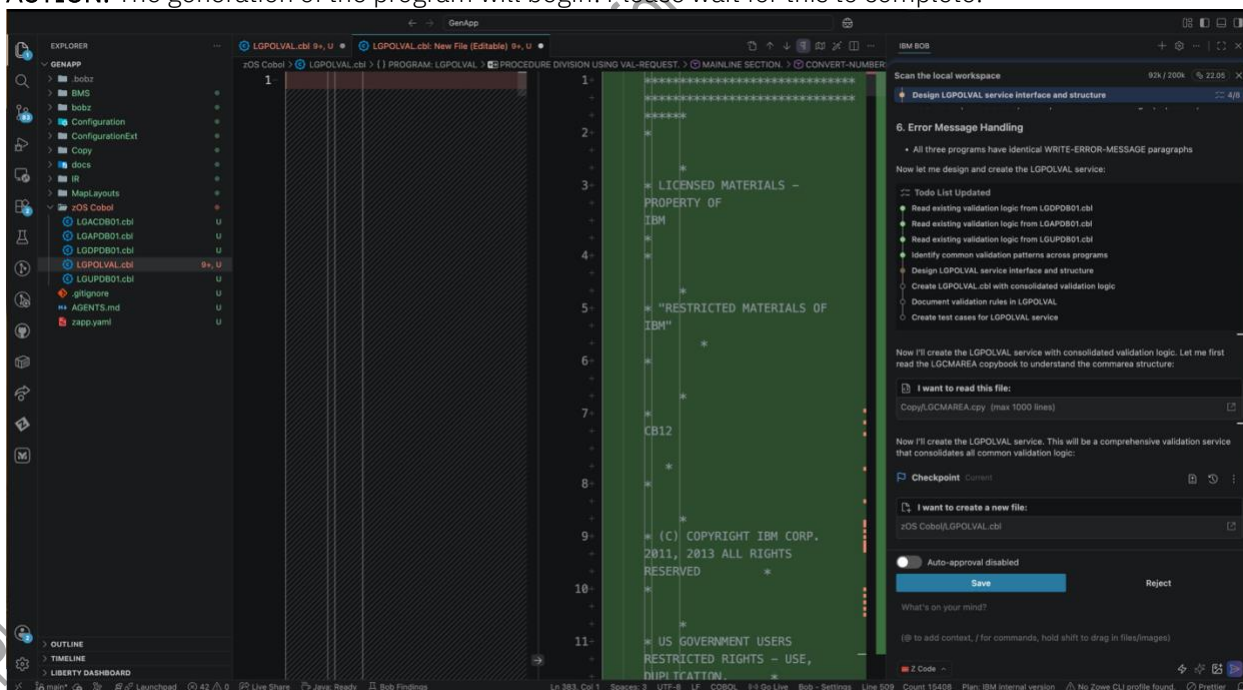
Create LGPOLVAL service that consolidates policy validation rules. Extract validation logic from LGDPDB01, LGAPDB01, LGAPVS01 and LGUPDB01 into the new service.



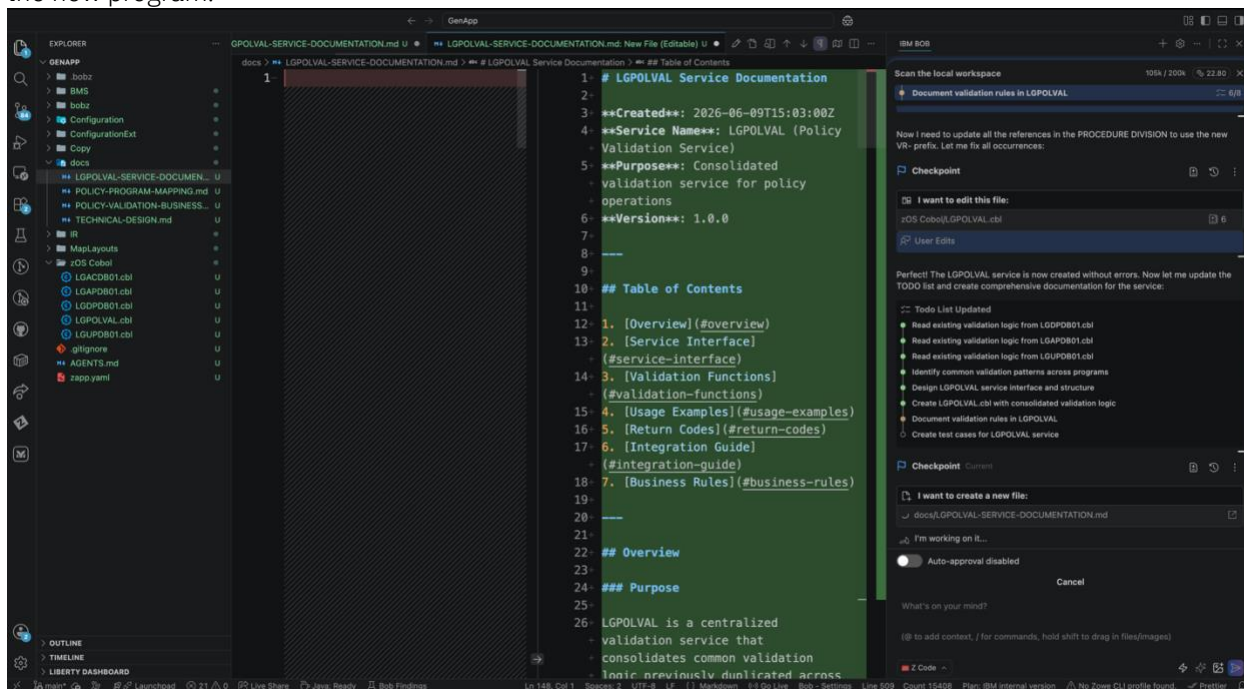
**ACTION:** BOB will generate a to-do list outlining all the changes it plans to make across the affected programs. Like before, you will receive a series of approvals. Please continue to select the **approve** button to proceed.



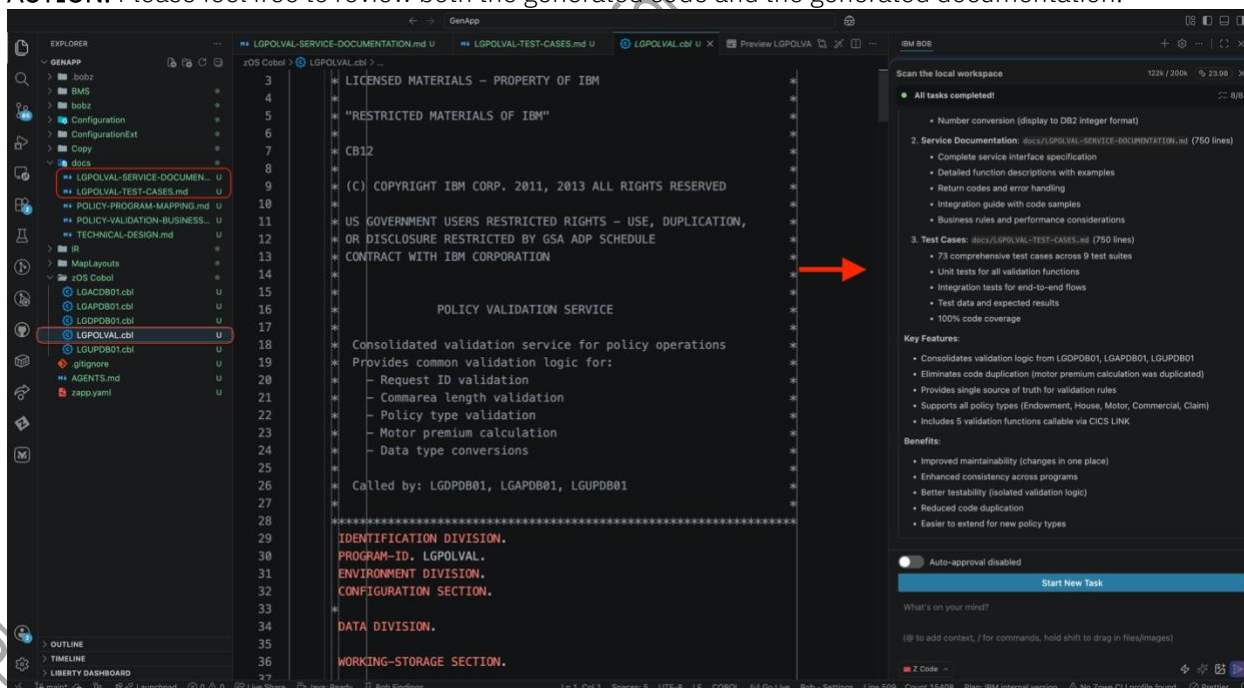
**ACTION:** The generation of the program will begin. Please wait for this to complete.



**ACTION:** You will also see additional documentation be generated, this corresponds to the completion of the new program.



**ACTION:** Please feel free to review both the generated code and the generated documentation.

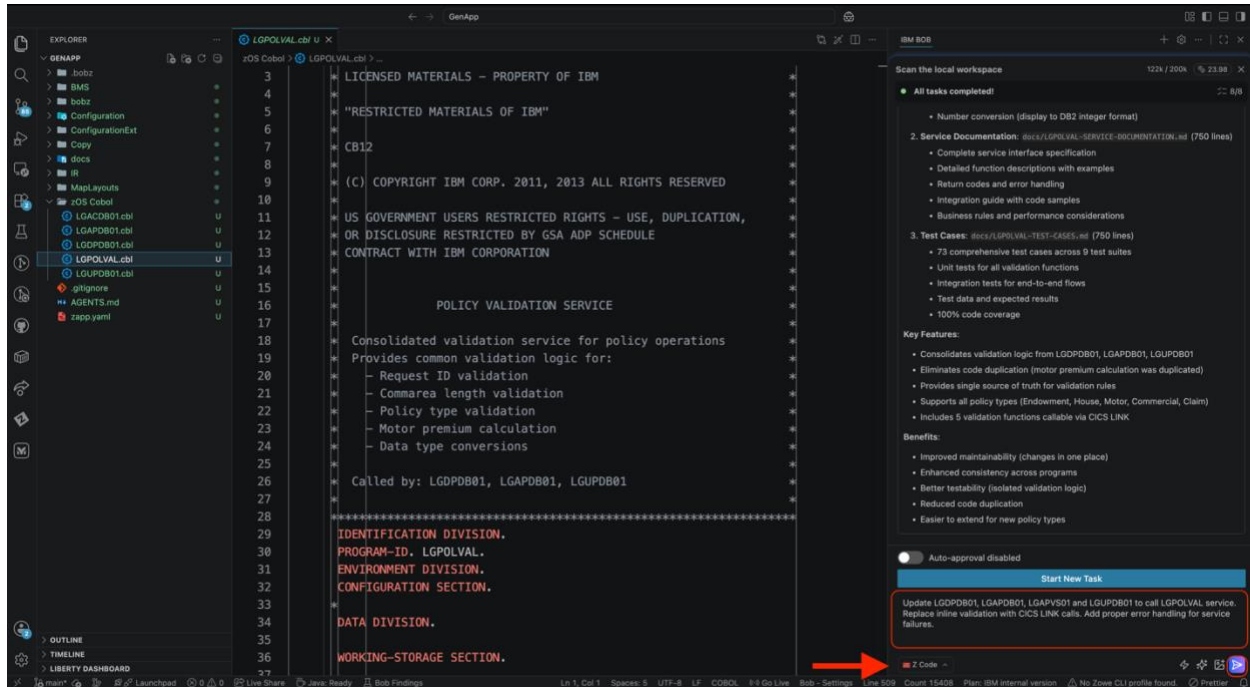




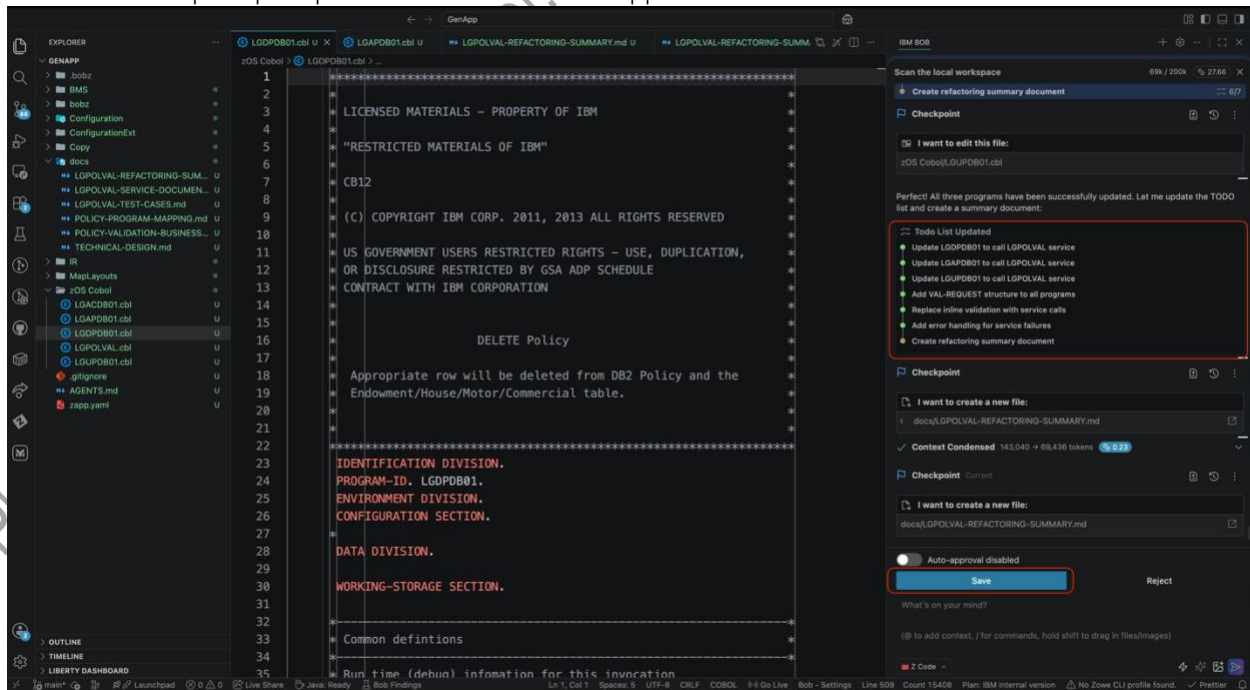
## Add link code to main module

**ACTION:** Please paste the following prompt on the chat window on the right as shown in the image below.

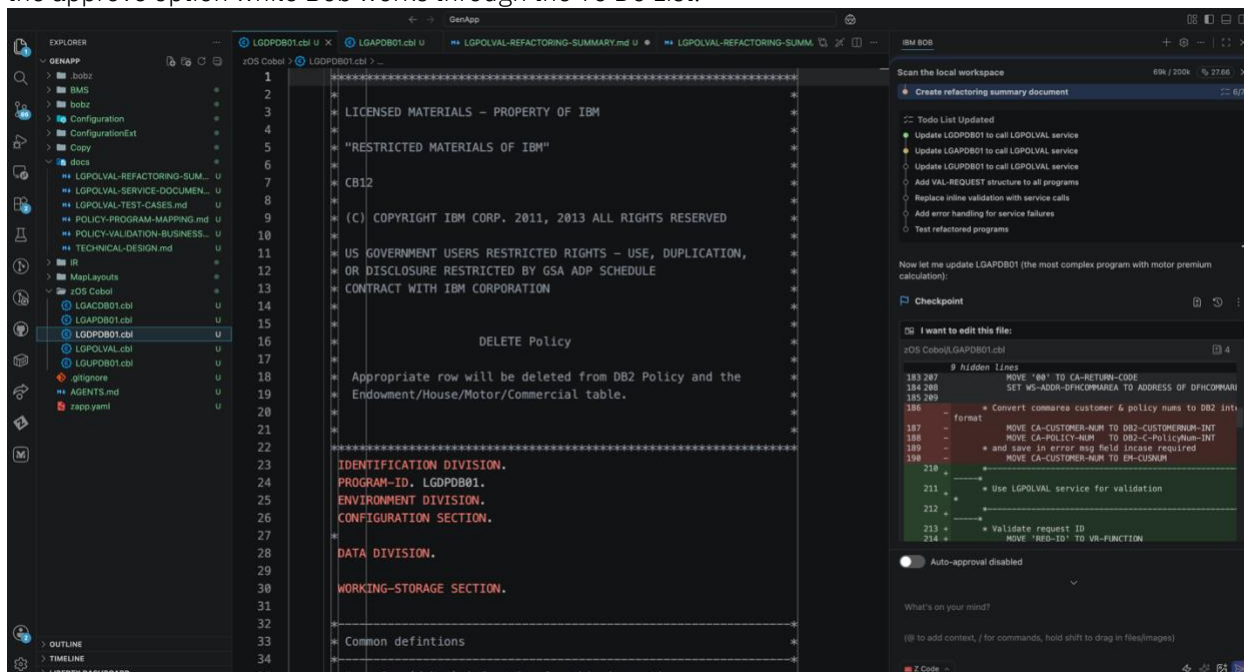
Update LGDPDB01, LGAPDB01, LGAPVS01 and LGUPDB01 to call LGPOLVAL service. Replace inline validation with CICS LINK calls. Add proper error handling for service failures.



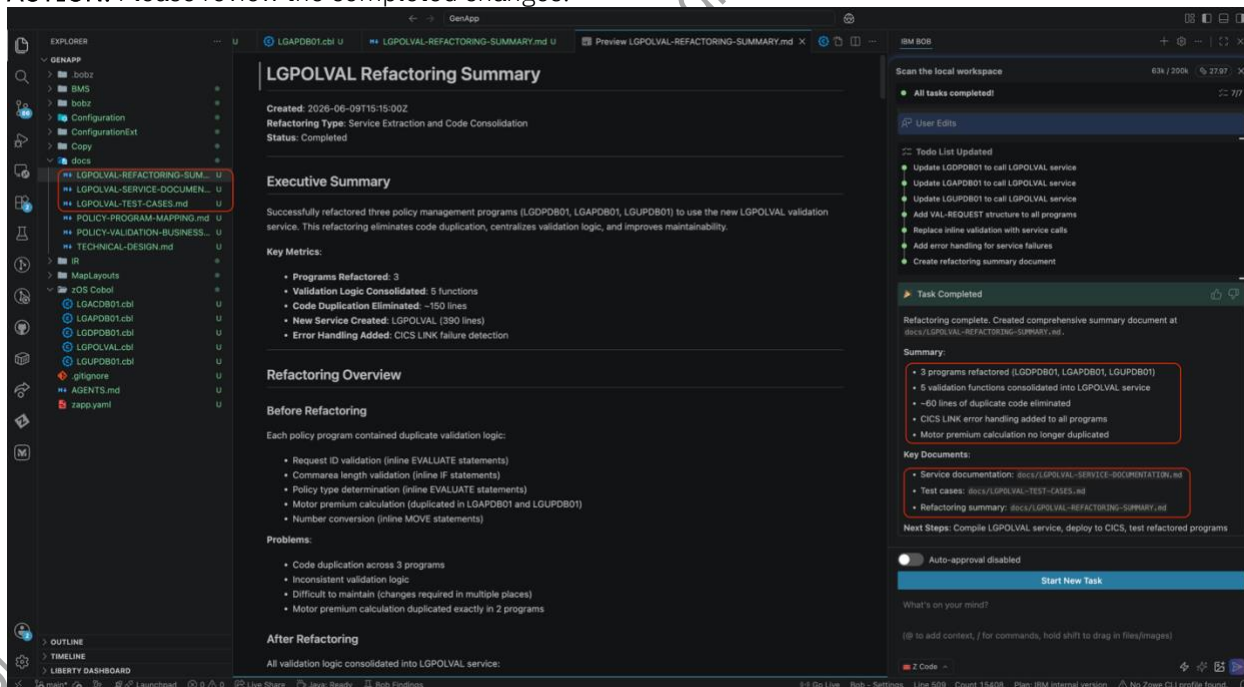
**ACTION:** When prompted please continue to select approve unless otherwise noted.



**ACTION:** You'll be able to see the exact changes while the to do list completes. Please continue to select the approve option while Bob works through the To Do List.



**ACTION:** Please review the completed changes!

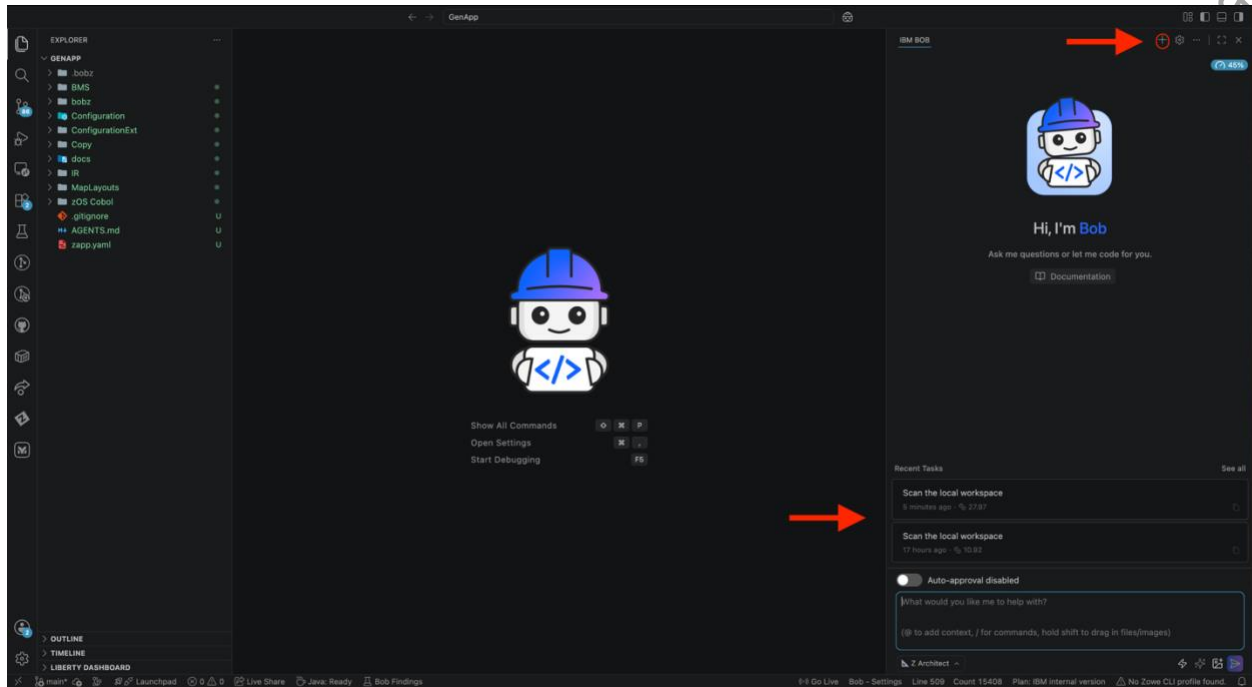




# CICS Online screens to modern UI Experience

## Understand & analyze insurance application

This section should begin with a fresh chat window. Please click the + to start a new chat window. This is also where you can browse your chat history if ever needed-where chat history follows the loaded code.



**ACTION:** In the chat box, please copy and paste the prompt below in **Z Architect** mode:

Create an analysis document of this application that helps analyze the conversion of greenscreens to a modern UI

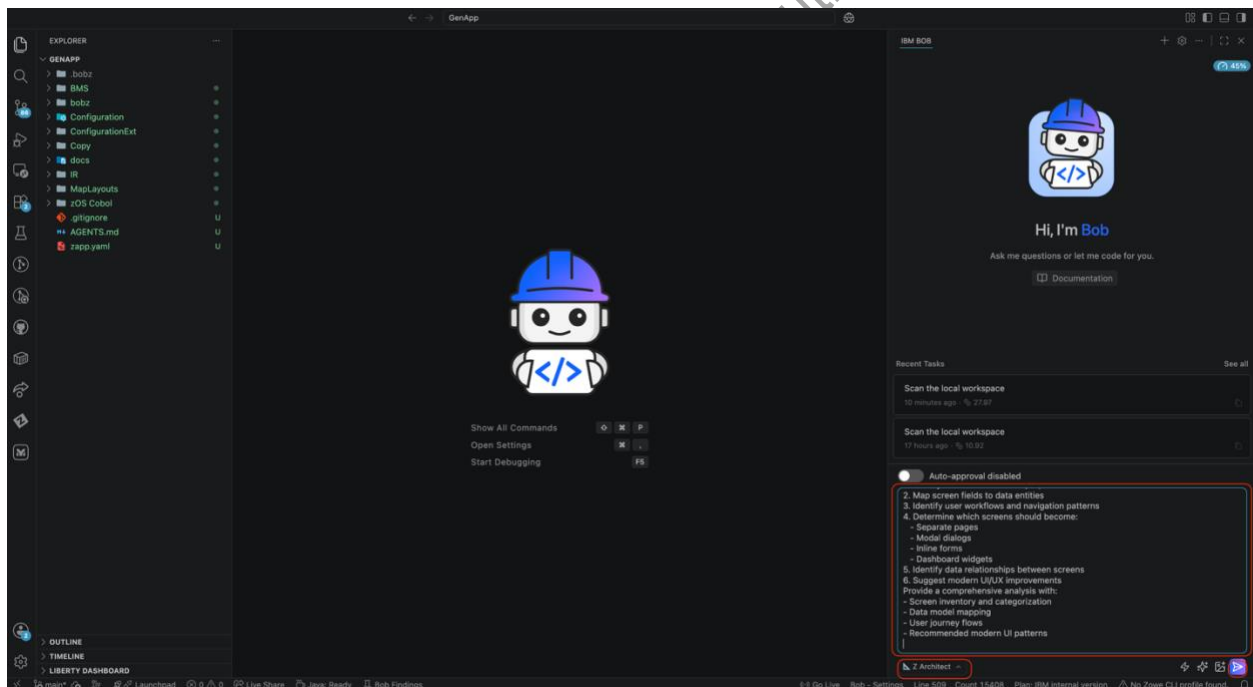
1. Identify all screens and their purposes
2. Map screen fields to data entities
3. Identify user workflows and navigation patterns
4. Determine which screens should become:

- Separate pages
- Modal dialogs
- Inline forms
- Dashboard widgets

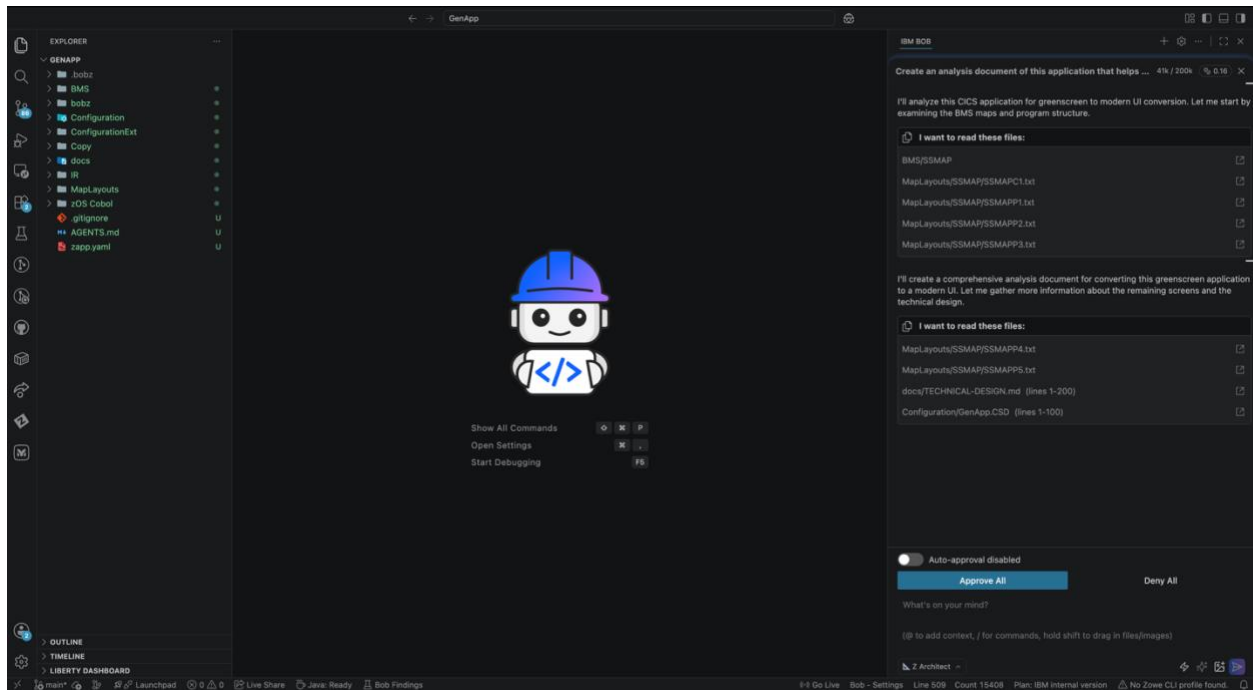
5. Identify data relationships between screens
6. Suggest modern UI/UX improvements

Provide a comprehensive analysis with:

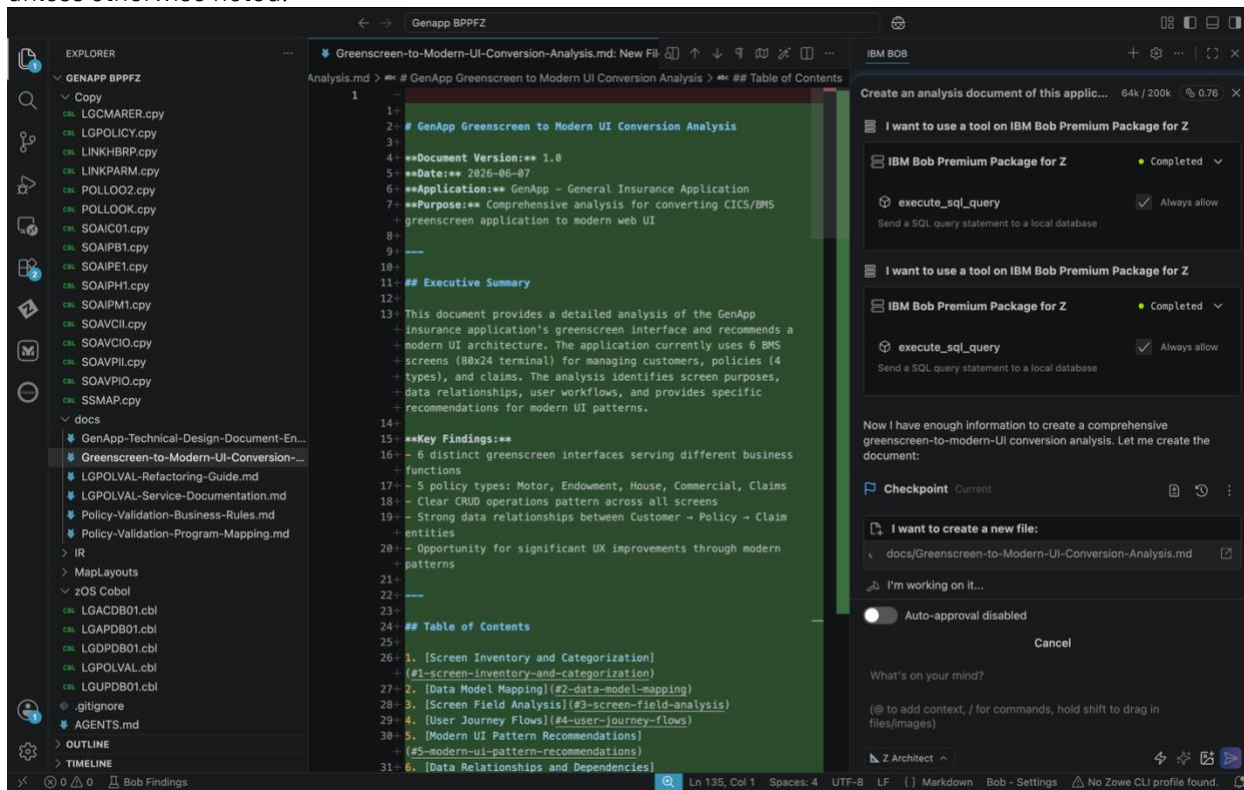
- Screen inventory and categorization
- Data model mapping
- User journey flows
- Recommended modern UI patterns



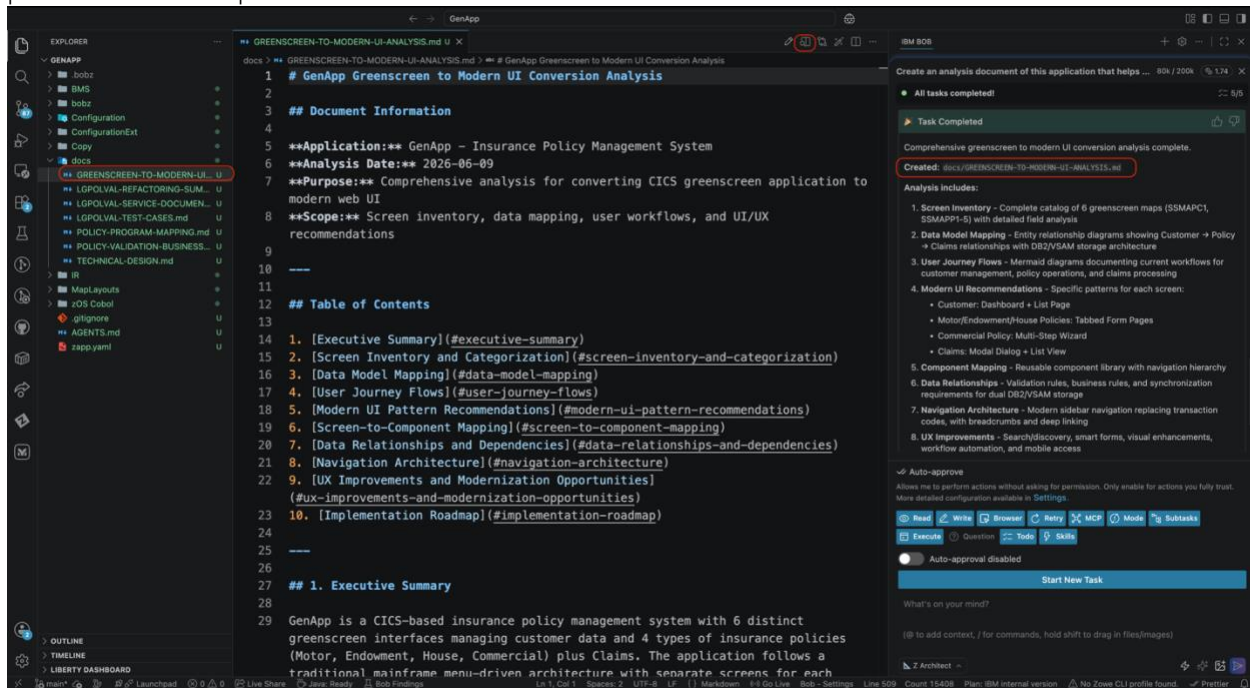
**ACTION:** In the chat box, please copy and paste the prompt below in **Z Architect** mode:  
You'll notice BOB will begin to generate information, such as reading from the various files to understand the ask.



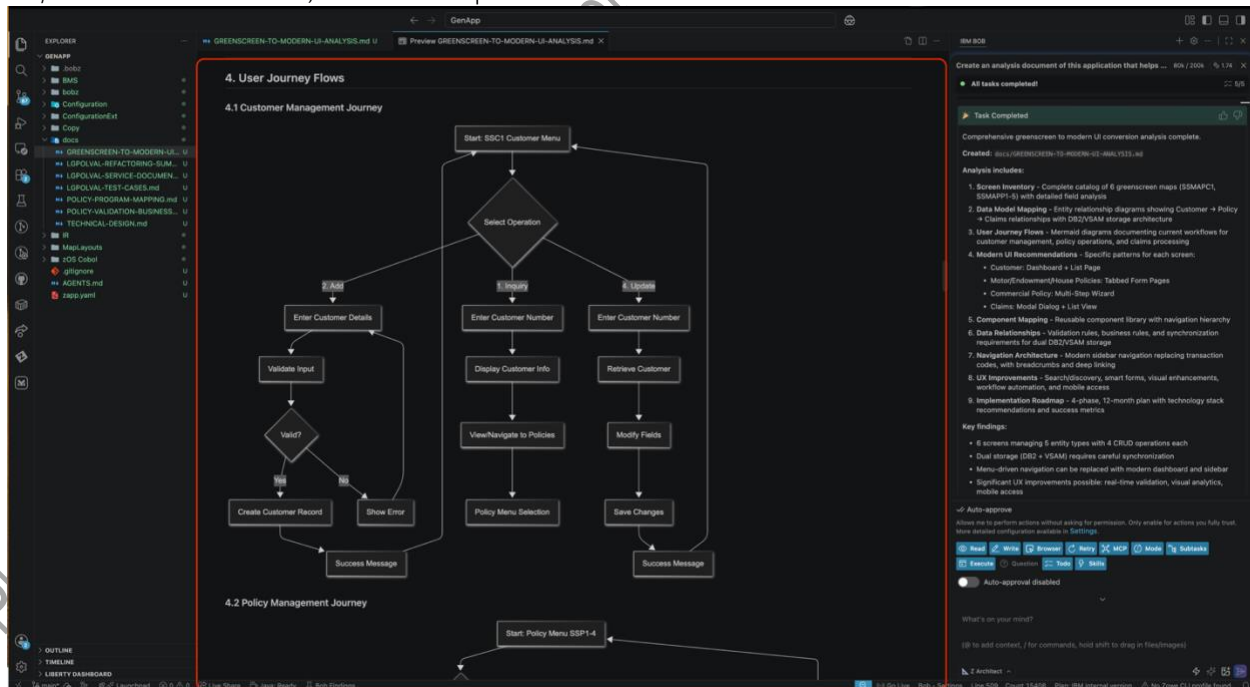
**ACTION:** In the chat box, please copy and paste the prompt below in **Z Architect** mode:  
Watch the generation complete in real time and if prompted for any other approvals please select yes unless otherwise noted.



**ACTION:** In the chat box, please copy and paste the prompt below in **Z Architect** mode:  
One or more documents may get generated. Once completed feel free to open the document(s) into preview mode to explore.



**ACTION:** Please scroll down the generated output to find information such as user journey flows, modern UX/UI Recommendations, Rest API Endpoints etc.





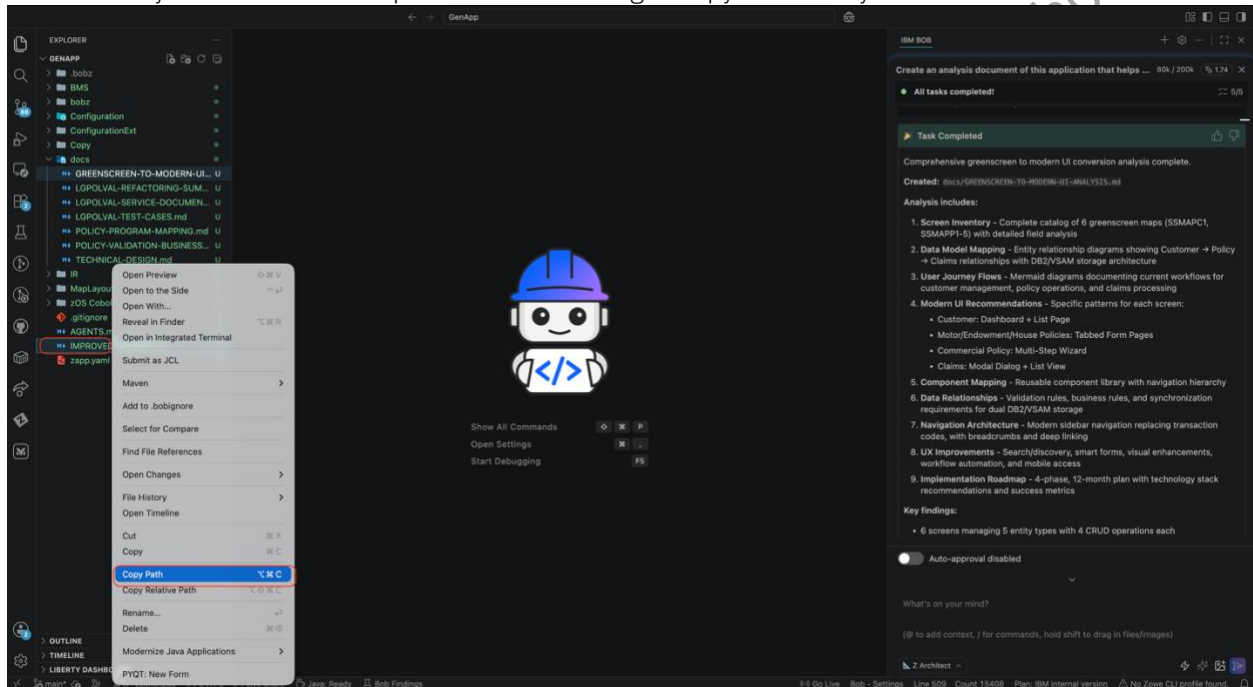
## Leverage prompt within chat to implement UI.

**ACTION:** Please switch to **Z Code Mode** and copy the path from the Improved\_Prompt.md to paste into the chat session.

In the chat window, enter the following prompt, replacing <path to file> with the actual file path of the Improved\_Prompt.md file. You can copy the path by right clicking the file in the explorer panel and selecting Copy Path:

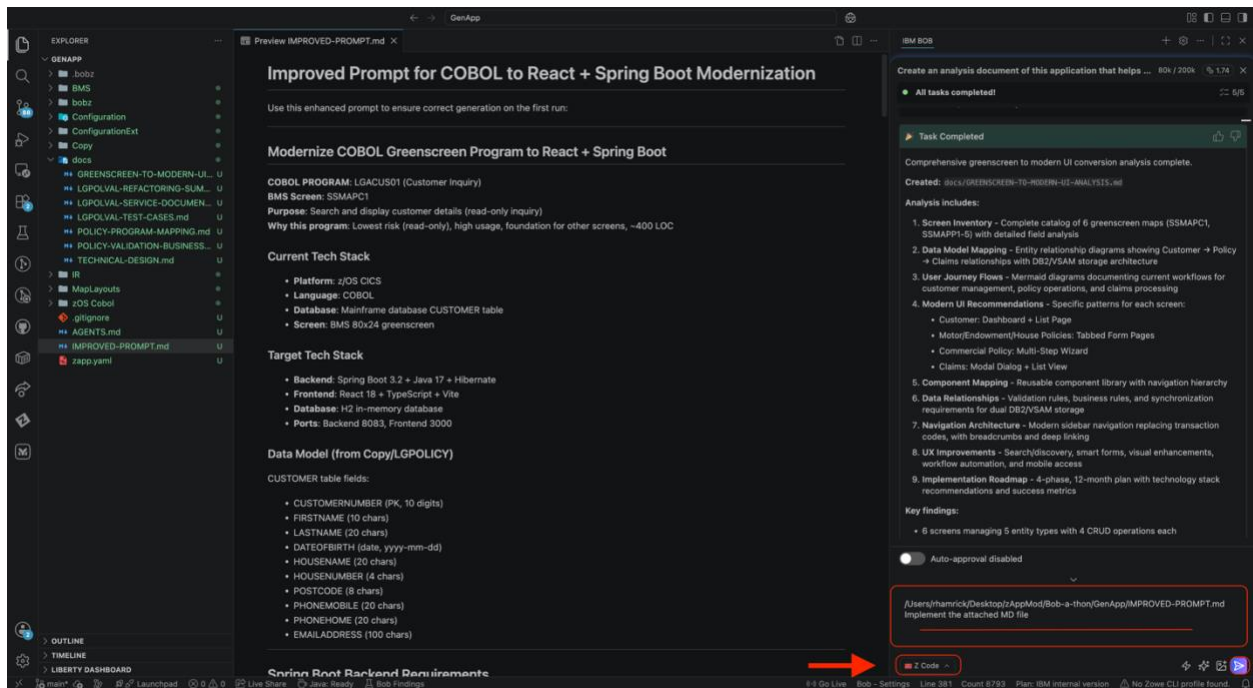
Use the <path to file> file to implement.

Tip: You can also use the @ symbol in the chat bar to reference files directly from the explorer, which automatically inserts the correct path without needing to copy it manually.

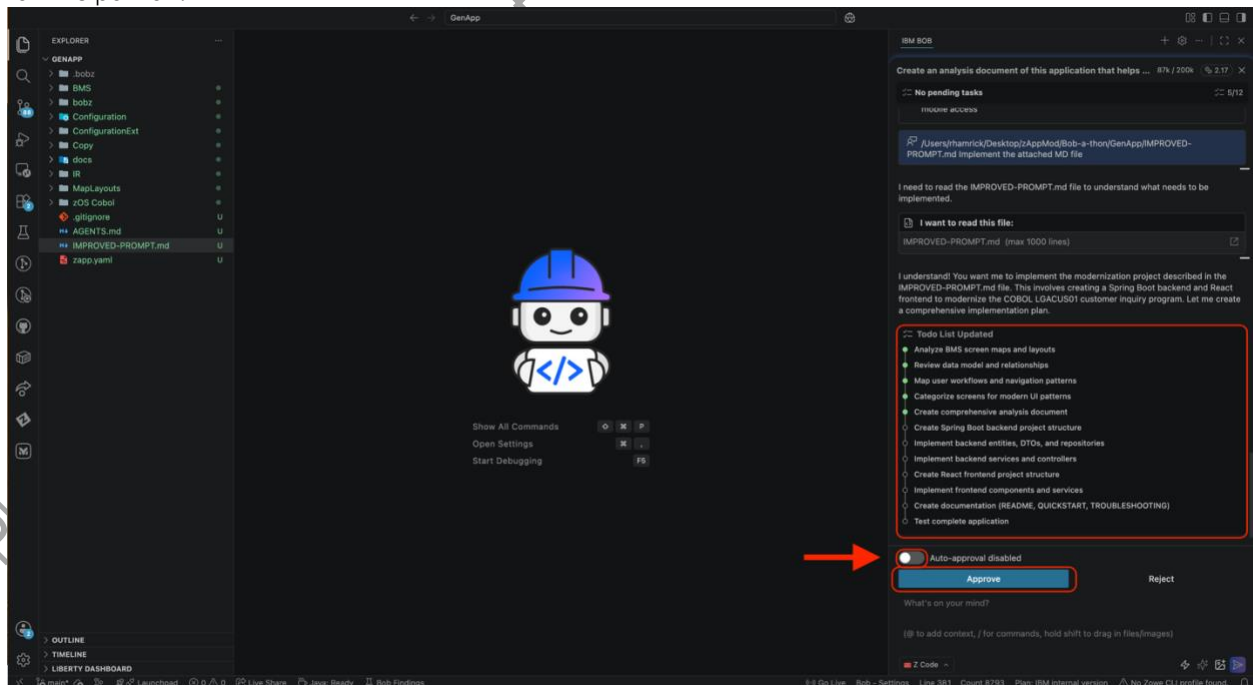


Action: For example:

<path> Implement the attached MD file



Action: You'll notice that BOB creates an action plan as it moves through the creation of these features. You will be asked to approve some actions if auto approve is turned off. Recommendation is to turn it on for this portion.





The screenshot shows a VS Code editor interface. On the left is the Explorer sidebar showing a project structure with folders like 'backend', 'src', 'resources', and 'components'. The main editor window displays a document titled 'Remaining Files to Complete Implementation'. The document content includes a heading 'Frontend Files Still Needed', a list of files to be created, and a code snippet for a 'CustomerDetail' component. The right sidebar shows a 'No pending tasks' section with a list of tasks and a 'Documentation' section with links to various documents. At the bottom, there is a status bar with information about the current file and project.

## Remaining Files to Complete Implementation

This document contains all remaining files needed to complete the Customer Inquiry modernization project.

### Frontend Files Still Needed

- frontend/src/components/CustomerDetail.tsx

```
import { Customer } from '../types/Customer';
import './CustomerDetail.css';

interface CustomerDetailProps {
  customer: Customer;
  onBack: () => void;
}

/**
 * Customer detail view component
 * Displays full customer information
 */
export function CustomerDetail({ customer, onBack }: CustomerDetailProps) {
  return (
    <div className="customer-detail">
      <button className="back-button" onClick={onBack}>
        Back to Search
      </button>

      <div className="detail-header">
        <h1>{customer.fullName}</h1>
        <span className="customer-number">Customer #{customer.customerNumber}</span>
      </div>

      <div className="detail-grid">
        <div className="detail-card">
          <h2>Personal Information</h2>
          <div className="detail-row">
            <span className="detail-label">First Name:</span>
            <span className="detail-value">{customer.firstName}</span>
          </div>
          <div className="detail-row">
            <span className="detail-label">Last Name:</span>
            <span className="detail-value">{customer.lastName}</span>
          </div>
          <div className="detail-row">
            <span className="detail-label">Date of Birth:</span>
            <span className="detail-value">{customer.dateOfBirth}</span>
          </div>
        </div>
      </div>
    </div>
  );
}
```

Create an analysis document of this application that helps...

No pending tasks

- Customer type definitions
- API service layer
- CustomerSearch component with autocomplete
- CustomerSearch CSS styling
- Project configuration files

Documentation:

- Comprehensive greenscreen-to-modern-UA analysis (BASIC/GREENSCREEN-TO-MODERN-UA-ANALYSIS.MD)
- Complete implementation guide with all remaining files (REMAINING-FILES-TO-CREATE.MD)

Remaining Files to Create:

The REMAINING-FILES-TO-CREATE.MD document contains complete code for:

- CustomerDetail.tsx and CSS
- App.tsx and CSS
- main.tsx
- index.css
- frontend/.gitignore
- README.md
- QUICKSTART.md
- TROUBLESHOOTING.md

To Complete Setup:

- Backend
  - cd backend
  - mvn spring-boot:run
- Frontend (new terminal)
  - cd frontend
  - npm install
  - npm run dev

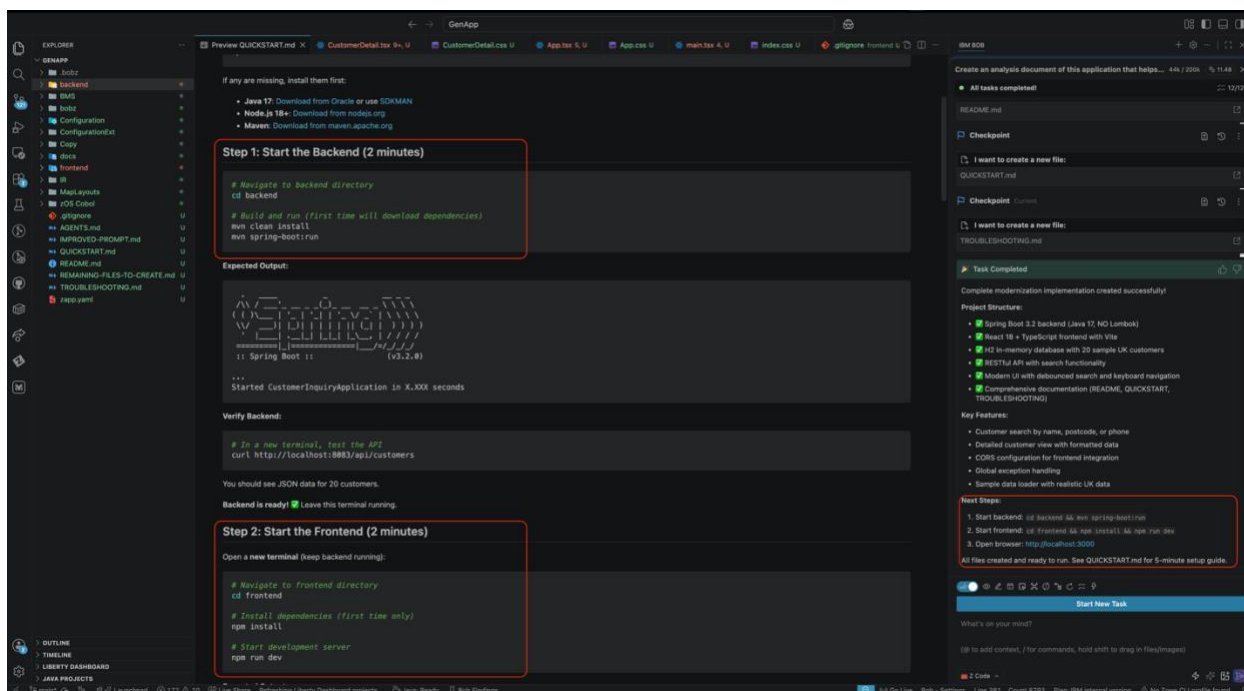
create the remaining files now

2 Code

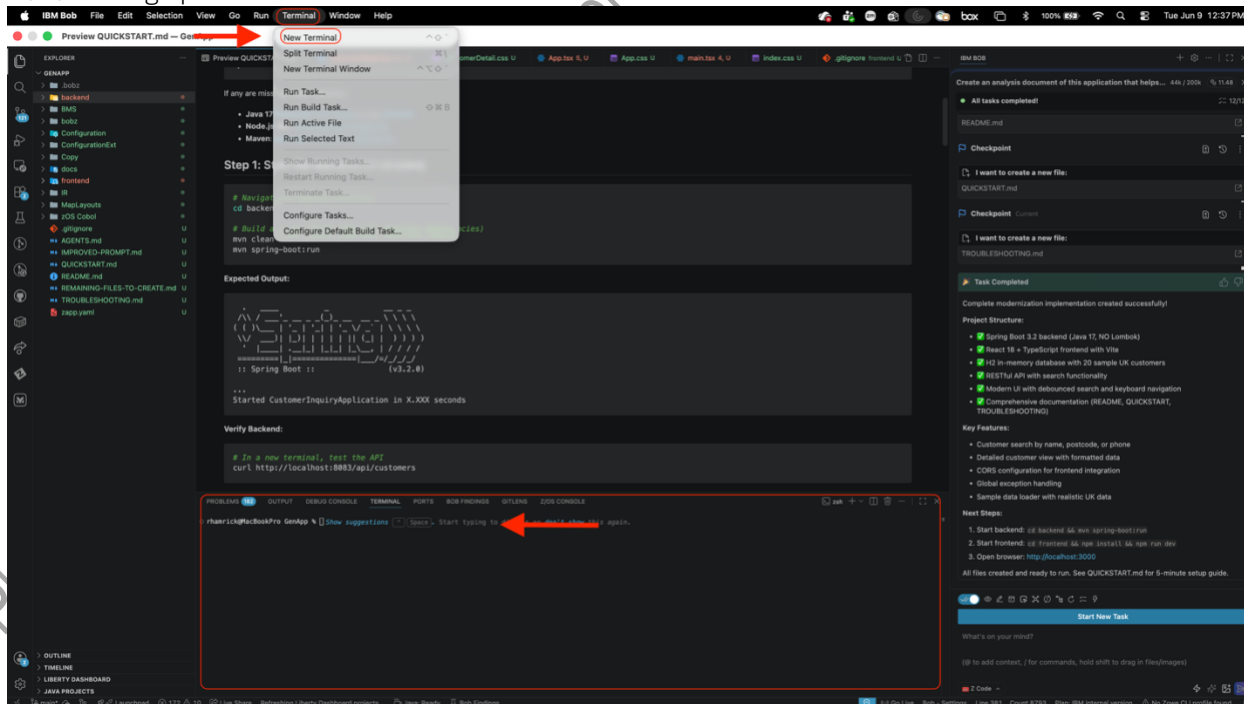
Go Live Bob - Settings Line 381 Count 8793 Plan: IBM internal version No Zowe CLJ profile found.

The screenshot displays the GenApp IDE interface. On the left, the Explorer panel shows the project structure with folders like 'backend', 'src', 'main', 'config', 'controller', 'dto', 'entity', 'repository', 'service', and 'types'. The 'backend' folder is highlighted. The main editor area shows a document titled 'Quick Start Guide - 5 Minutes to Running Application'. The document content includes a 'Prerequisites Check' section with instructions to verify Java 17, Node.js 18+, Maven 3.6+, and npm versions. Below this, it lists where to download these tools if missing. The 'Step 1: Start the Backend (2 minutes)' section provides commands to navigate to the backend directory, build, and run the application. The terminal window at the bottom shows the execution of these commands: `cd backend`, `mvn clean install`, and `mvn spring-boot:run`. The output shows the application starting successfully on port 3000.

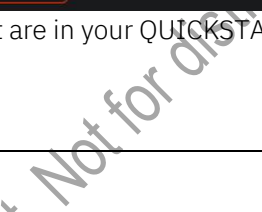
**Action:** You will see some commands to start the services. These commands may be slightly different from the screenshots below. If you receive an error message in starting a service, please flag an instructor down.



**Action:** Bring up terminal



Approval



are in your QUICKSTART

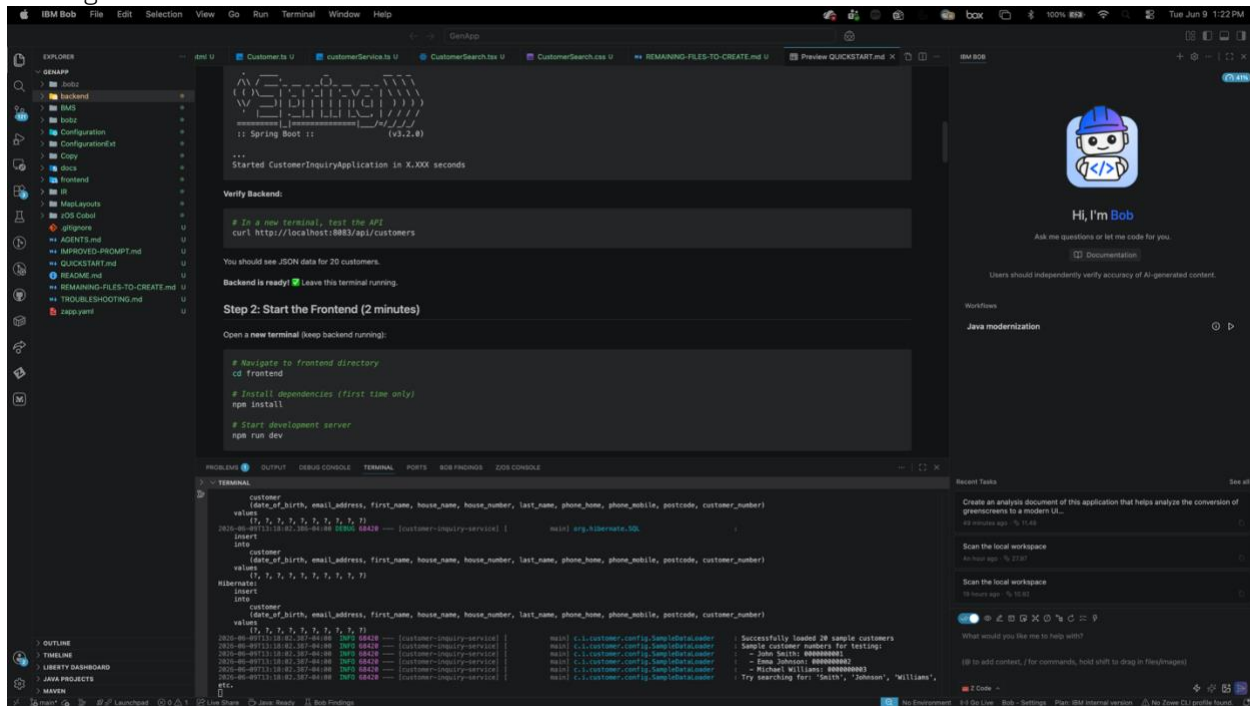
are in your QUICKSTART

are in your QUICKSTART





**ACTION:** You should see where it mentions having loaded 20 sample customers. Leave this terminal running.



```
...
Started CustomerInquiryApplication in X.XXX seconds

Verify Backend:

# In a new terminal, test the API
curl http://localhost:8083/api/customers

You should see JSON data for 20 customers.

Backend is ready! Leave this terminal running.

Step 2: Start the Frontend (2 minutes)

Open a new terminal (keep backend running):

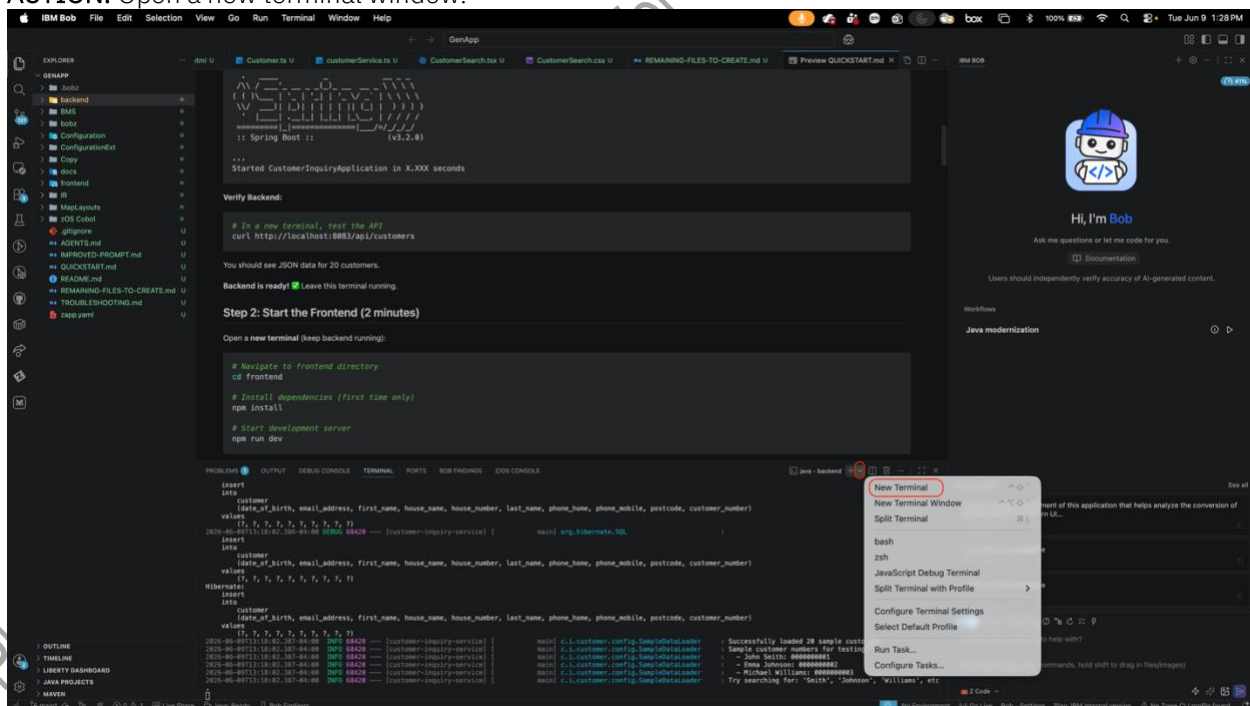
# Navigate to frontend directory
cd frontend

# Install dependencies (first time only)
npm install

# Start development server
npm run dev

...
2025-06-09T13:18:02.387-04:00 INFO 08428 [customer-inquiry-service] main c:\customer.config.SampleDataLoader Successfully loaded 20 sample customers
2025-06-09T13:18:02.387-04:00 INFO 08428 [customer-inquiry-service] main c:\customer.config.SampleDataLoader Sample customer numbers for testing:
2025-06-09T13:18:02.387-04:00 INFO 08428 [customer-inquiry-service] main c:\customer.config.SampleDataLoader - Jane Smith: 8000000001
2025-06-09T13:18:02.387-04:00 INFO 08428 [customer-inquiry-service] main c:\customer.config.SampleDataLoader - Rene Johnson: 8000000002
2025-06-09T13:18:02.387-04:00 INFO 08428 [customer-inquiry-service] main c:\customer.config.SampleDataLoader - Michael Williams: 8000000003
2025-06-09T13:18:02.387-04:00 INFO 08428 [customer-inquiry-service] main c:\customer.config.SampleDataLoader Try searching for: 'Smith', 'Johnson', 'Williams', ...
```

**ACTION:** Open a new terminal window.



```
...
Started CustomerInquiryApplication in X.XXX seconds

Verify Backend:

# In a new terminal, test the API
curl http://localhost:8083/api/customers

You should see JSON data for 20 customers.

Backend is ready! Leave this terminal running.

Step 2: Start the Frontend (2 minutes)

Open a new terminal (keep backend running):

# Navigate to frontend directory
cd frontend

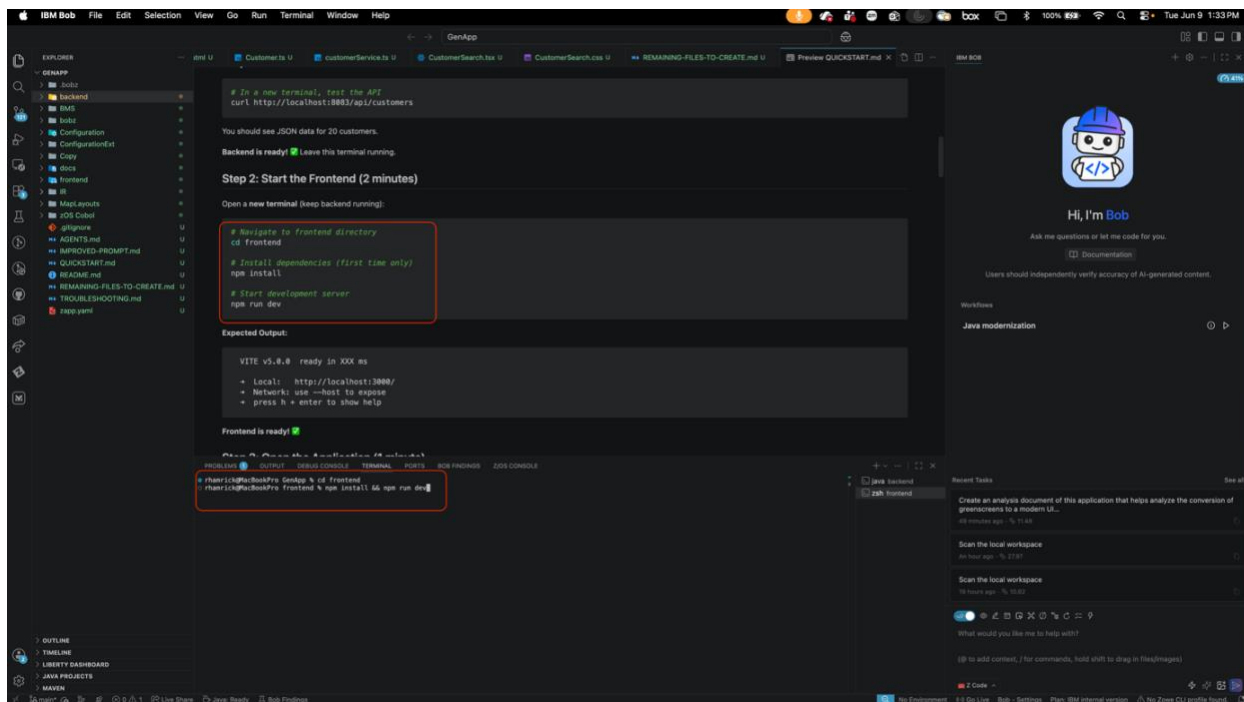
# Install dependencies (first time only)
npm install

# Start development server
npm run dev

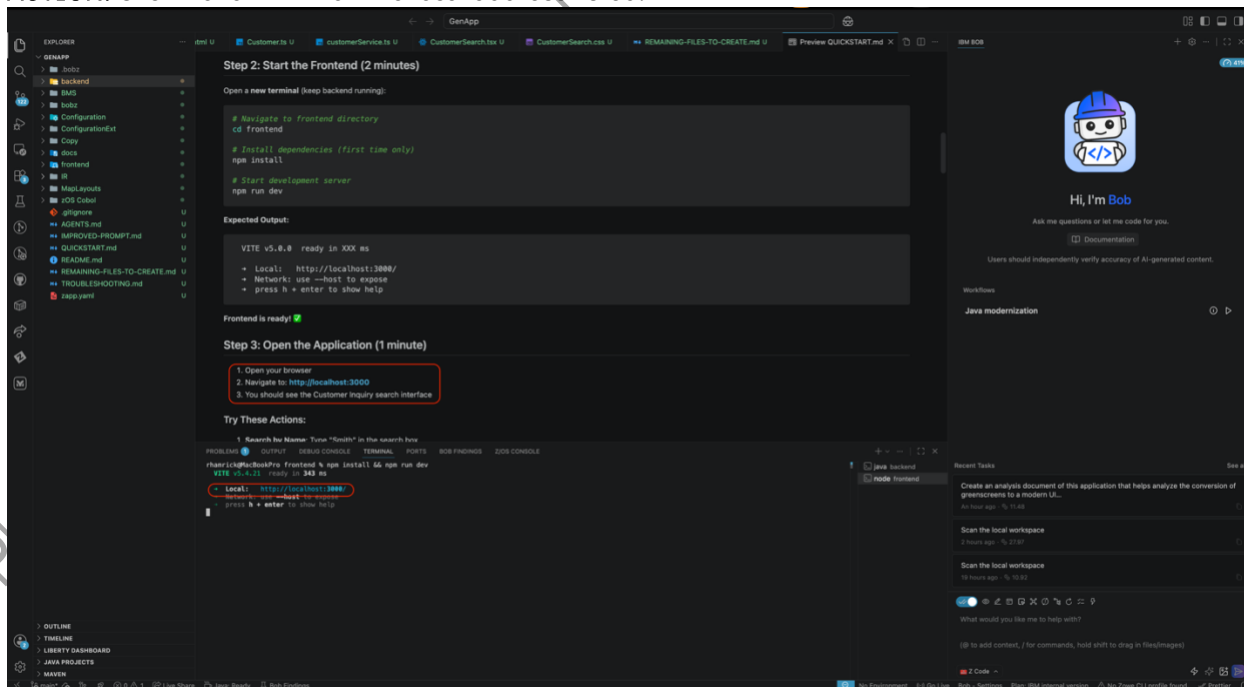
...
2025-06-09T13:18:02.387-04:00 INFO 08428 [customer-inquiry-service] main c:\customer.config.SampleDataLoader Successfully loaded 20 sample customers
2025-06-09T13:18:02.387-04:00 INFO 08428 [customer-inquiry-service] main c:\customer.config.SampleDataLoader Sample customer numbers for testing:
2025-06-09T13:18:02.387-04:00 INFO 08428 [customer-inquiry-service] main c:\customer.config.SampleDataLoader - Jane Smith: 8000000001
2025-06-09T13:18:02.387-04:00 INFO 08428 [customer-inquiry-service] main c:\customer.config.SampleDataLoader - Rene Johnson: 8000000002
2025-06-09T13:18:02.387-04:00 INFO 08428 [customer-inquiry-service] main c:\customer.config.SampleDataLoader - Michael Williams: 8000000003
2025-06-09T13:18:02.387-04:00 INFO 08428 [customer-inquiry-service] main c:\customer.config.SampleDataLoader Try searching for: 'Smith', 'Johnson', 'Williams', ...
```

ACTION: Issue the following commands:

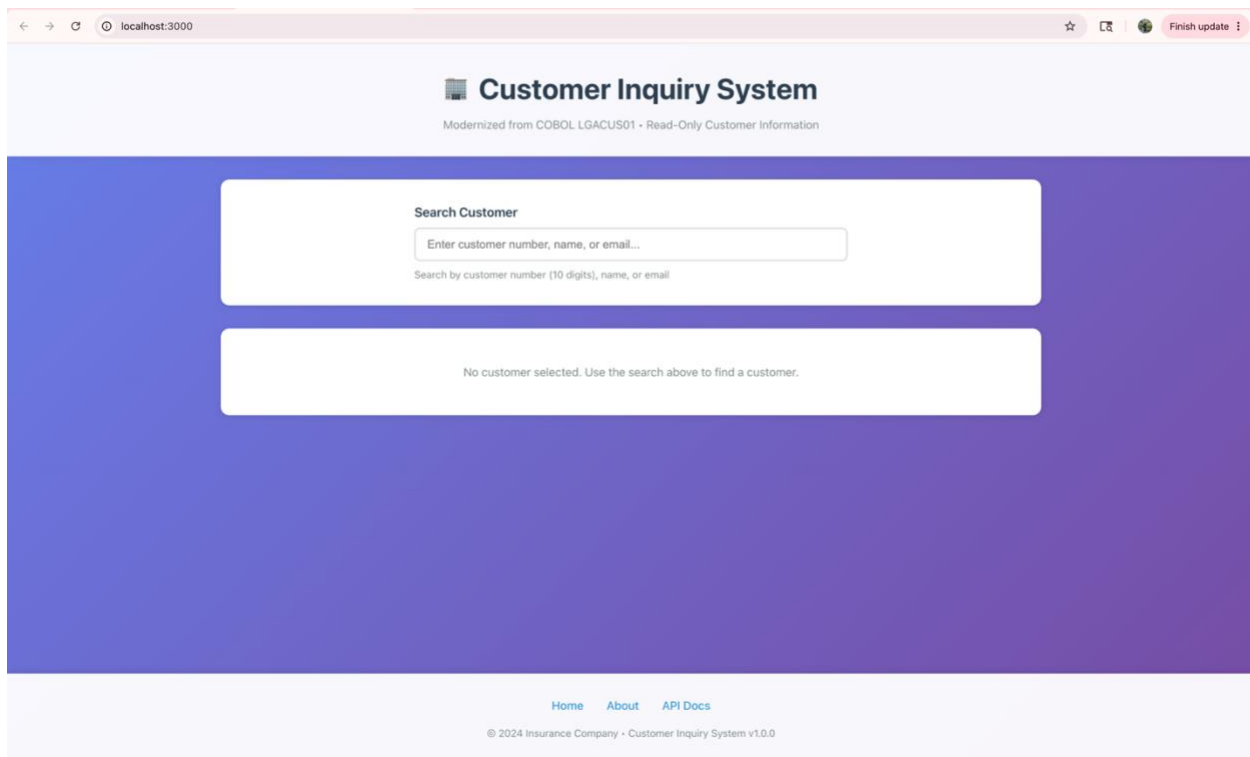
```
Cd <path to frontend>
Npm install && npm run dev
```



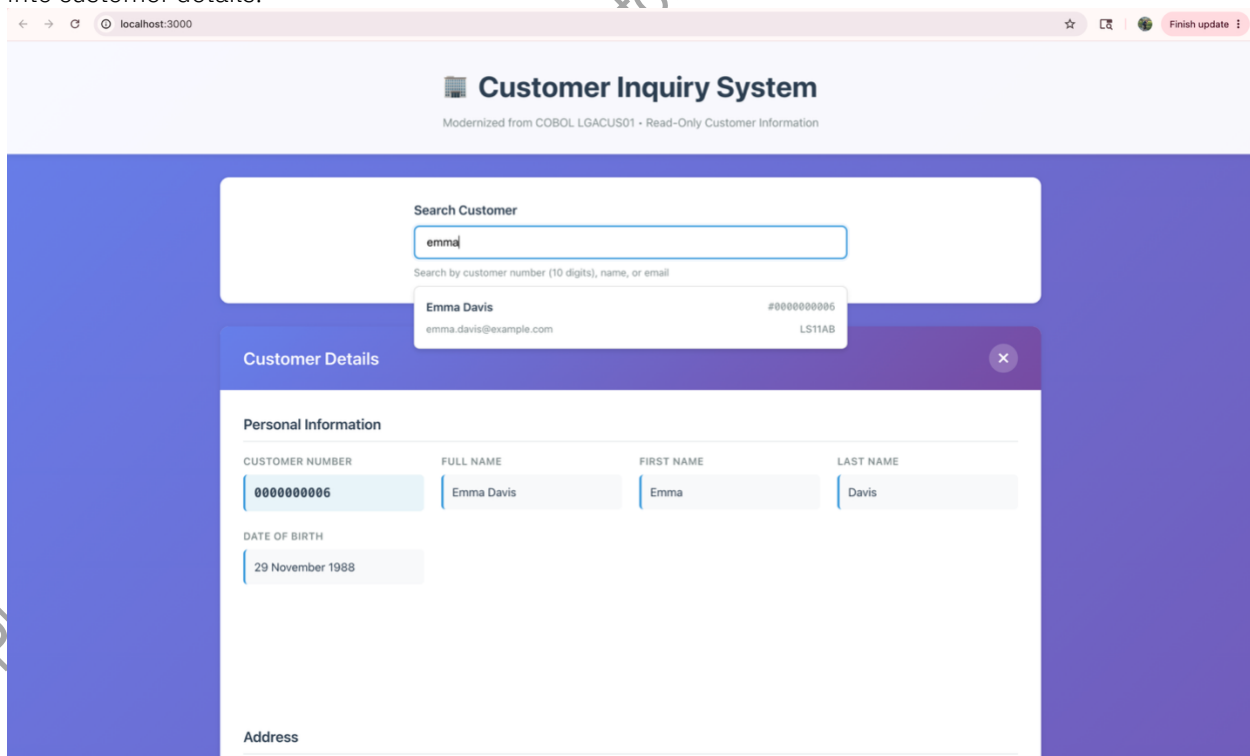
ACTION: Click “follow link” on the local address listed.



ACTION: Explore UI



**ACTION:** Try typing Emma into the search bar, here we can see that personal information as been added into customer details.



⚠️ If your UI doesn't look like the one in the picture you can grab a screenshot and prompt Bob in code mode to make your frontend to look like the picture attached and add the picture and he will match it.